

# Pruebas del Software

---

Este documento explica cómo abordar la fase de pruebas del ciclo de vida del software. Se verán distintas técnicas de diseño de casos de prueba. Además, se empleará un IDE para la depuración de código. Finalmente, emplearemos una herramienta para la realización de pruebas automatizadas.

---

## Pruebas del Software

Copyright © 2023 by Rafael Lozano Luján.

Este documento está sujeto a derechos de autor. Todos los derechos están reservados por el Autor de la obra, ya sea en su totalidad o en parte de ella, específicamente los derechos de:

- La reproducción total o parcial mediante fotocopia, escaneo o descarga.
- La distribución o publicación de copias a terceros.
- La transformación mediante la modificación, traducción o adaptación que altere la forma de la obra hasta obtener una diferente a la original.

## Tabla de contenido

|         |                                                |    |
|---------|------------------------------------------------|----|
| 1       | Introducción.....                              | 1  |
| 2       | Estrategias de pruebas del software.....       | 3  |
| 2.1     | Prueba unitarias.....                          | 4  |
| 2.2     | Prueba de integración.....                     | 5  |
| 2.3     | Prueba de sistema.....                         | 5  |
| 2.4     | Prueba de validación.....                      | 6  |
| 2.5     | Prueba de regresión.....                       | 6  |
| 3       | Diseño de casos de prueba.....                 | 7  |
| 3.1     | Pruebas de la caja blanca o estructurales..... | 7  |
| 3.1.1   | Prueba del camino básico.....                  | 8  |
| 3.1.2   | Prueba de bucles.....                          | 14 |
| 3.1.3   | Prueba de flujo de datos.....                  | 15 |
| 3.2     | Pruebas de la caja negra o funcionales.....    | 17 |
| 3.2.1   | Particiones o clases de equivalencia.....      | 17 |
| 3.2.2   | Análisis de valores límite.....                | 20 |
| 3.2.3   | Conjetura de errores.....                      | 21 |
| 3.3     | El proceso de pruebas del software.....        | 22 |
| 4       | Documentación de las pruebas.....              | 22 |
| 4.1     | Diseño de los casos de prueba.....             | 22 |
| 4.2     | Ejecución de las pruebas.....                  | 23 |
| 5       | Calidad del software.....                      | 25 |
| 5.1     | Medidas de calidad del software.....           | 25 |
| 5.1.1   | Métricas del producto.....                     | 26 |
| 5.1.2   | Métricas del proceso.....                      | 26 |
| 5.1.3   | Métricas del proyecto.....                     | 26 |
| 5.2     | Normas y certificaciones.....                  | 27 |
| 6       | Depuración de código.....                      | 28 |
| 6.1     | PyCharm.....                                   | 29 |
| 6.1.1   | Puntos de ruptura.....                         | 29 |
| 6.1.2   | Comenzar la sesión de depuración.....          | 32 |
| 6.1.3   | Examinar un programa suspendido.....           | 33 |
| 6.1.3.1 | Examinar marcos.....                           | 33 |
| 6.1.3.2 | Examinar variables.....                        | 34 |
| 6.1.3.3 | Evaluar expresiones.....                       | 36 |
| 6.1.3.4 | Ver valores en línea.....                      | 37 |
| 6.1.3.5 | Observaciones.....                             | 39 |
| 6.1.3.6 | Regresar al punto de ejecución.....            | 39 |
| 6.1.4   | Ejecución paso a paso.....                     | 39 |
| 6.1.4.1 | Step over.....                                 | 40 |
| 6.1.4.2 | Step into.....                                 | 40 |

|                                                   |    |
|---------------------------------------------------|----|
| 6.1.4.3 Step into my code.....                    | 40 |
| 6.1.4.4 Smart step into.....                      | 40 |
| 6.1.4.5 Step out.....                             | 41 |
| 6.1.4.6 Run to cursor.....                        | 41 |
| 6.1.4.7 Force run to cursor.....                  | 41 |
| 6.1.4.8 Force step over.....                      | 41 |
| 6.1.4.9 Jump to cursor.....                       | 42 |
| 6.2 VS Code.....                                  | 42 |
| 6.2.1 Puntos de ruptura y puntos de registro..... | 43 |
| 6.2.2 Ejecución paso a paso.....                  | 44 |
| 6.2.3 Inspección de datos.....                    | 44 |
| 6.2.4 Consola depuración REPL.....                | 46 |
| 7 Pruebas automatizadas con UnitTest.....         | 46 |
| 7.1 Ejemplo de prueba.....                        | 47 |
| 8 Bibliografía.....                               | 53 |

# Pruebas del Software

## 1 Introducción

---

Cuando se desarrolla una aplicación, esta tiene que responder a los requisitos especificados por el cliente y debe tener el menor número posible de errores. Para ello, la etapa de pruebas del ciclo de vida del software nos permite descubrir dichos defectos antes de que el software sea entregado al cliente.

Las pruebas tienen como objetivo fundamental la detección de defectos que se han cometido durante el proceso de desarrollo del software. El punto de partida es una premisa importante:

*Todo software desarrollado tiene defectos. Las pruebas no tienen que demostrar que el software no los tiene, sino todo lo contrario.*

Aunque estemos desarrollando una pequeña aplicación, es imposible probar exhaustivamente el software, es decir, es imposible encontrar todos sus errores. Por tanto tendremos que encontrar un equilibrio entre el esfuerzo necesario para detectar los errores en el software y los errores detectados.

La importancia de esta tarea radica en que todo error no detectado en esta fase estará presente en el software entregado al cliente, y hay una probabilidad muy alta de que sea el propio cliente el que lo detecte, algo indeseable que daña la imagen del software y del que lo ha desarrollado. Entre los programadores informáticos hay una creencia muy extendida y verificada:

*El cliente tiene la capacidad y habilidad de detectar errores que no han sido detectados durante la etapa de pruebas.*

Otra cuestión importante es que, siempre que sea posible, el desarrollador no debería participar en las pruebas del software, excepto en las pruebas unitarias que realiza conforme desarrolla la aplicación. La fase de pruebas tiene un carácter destructivo, es decir, hay que someter al software a una serie de pruebas que revelarán fallos en la etapa de desarrollo. De ahí que, como cualquier persona, el programador no sea muy proclive a someter el software que ha desarrollado a unas pruebas exhaustivas que podrían detectar numerosos fallos, lo cual es un indicativo de la calidad del software.

La realización de las pruebas al software cubre dos aspectos:

- ✓ Verificación del software → Comprobar si el producto que se está desarrollando funciona correctamente, es decir, no contiene errores de programación que impidan al software ejecutarse.
- ✓ Validación del software → Consiste en comprobar si se está desarrollando el software correcto, es decir, de acuerdo a los requisitos del cliente.

Por tanto, las pruebas deben comprobar tanto si el producto es el que quiere el cliente (validación) como si está desarrollado correctamente, de manera que lo que hace, lo hace bien (verificación).

Además, hay aspectos sobre la estrategia de las pruebas del software a seguir relacionada con la puesta en práctica:

- ✓ La estrategia de aplicación de las pruebas, es decir, decidir qué partes del software deben someterse a pruebas a medida que se va desarrollando.
- ✓ Las técnicas de diseño de casos de prueba que se van a emplear. Una vez seleccionada la parte del software que se someterá a prueba, se establece qué técnica se emplea para llevarla a cabo.

¿Qué es un caso de prueba? El IEEE (*Institute of Electrical and Electronics Engineers*) ha definido el concepto de caso de prueba de la siguiente forma:

*«un conjunto de datos de entrada, condiciones de ejecución y resultados esperados desarrollados para un objetivo particular como, por ejemplo, ejercitar un camino concreto de un programa o verificar el cumplimiento de un determinado requisito».*

Un caso de prueba consiste básicamente en:

- ✓ Indicar qué datos de entrada se deben suministrar al programa, qué datos de salida se deben obtener y cuál es el fin de llevar a cabo dicho caso de prueba.
- ✓ Una vez ejecutada la prueba, comprobar si la salida obtenida coincide con la salida esperada. La coincidencia indicará que no se ha detectado defecto alguno para ese

caso de prueba. En caso contrario, se realiza una tarea conocida con el nombre de depuración.

La depuración es, por tanto, la consecuencia de una prueba exitosa (es decir, ha encontrado un defecto en el software) y consiste en localizar el error o defecto en el programa y corregirlo, para lo que podría ser necesario ejecutar nuevos casos de prueba.

Una máxima establecida por Piattini respecto a la hora de afrontar las pruebas es:

*Un buen caso de prueba es aquel que tiene una gran probabilidad de encontrar un defecto no descubierto aún y que el éxito de una prueba consiste en detectar un defecto no encontrado antes.*

Este criterio es el que debe guiar el proceso de prueba del software. En proyectos de tamaño medio/grande es habitual que esta tarea la realiza un equipo de pruebas independiente que trabajan conjuntamente con las personas encargadas del desarrollo del software para garantizar que se realicen pruebas exhaustivas del programa.

## 2 Estrategias de pruebas del software

Las pruebas siempre comienzan por los componentes más pequeños del software, es decir, se empieza probando funciones o métodos de clase, y se va incrementando de manera progresiva el alcance y ámbito de la prueba. Se suelen llevar a cabo cuatro tipos de pruebas secuenciales que se exponen a continuación.

Todos estos tipos de pruebas se muestran en el modelo en V, que es una variante del modelo en cascada. En su representación, los diferentes niveles de pruebas se colocan formando una V para mostrar la relación de cada fase del ciclo de vida con su fase de pruebas asociada.

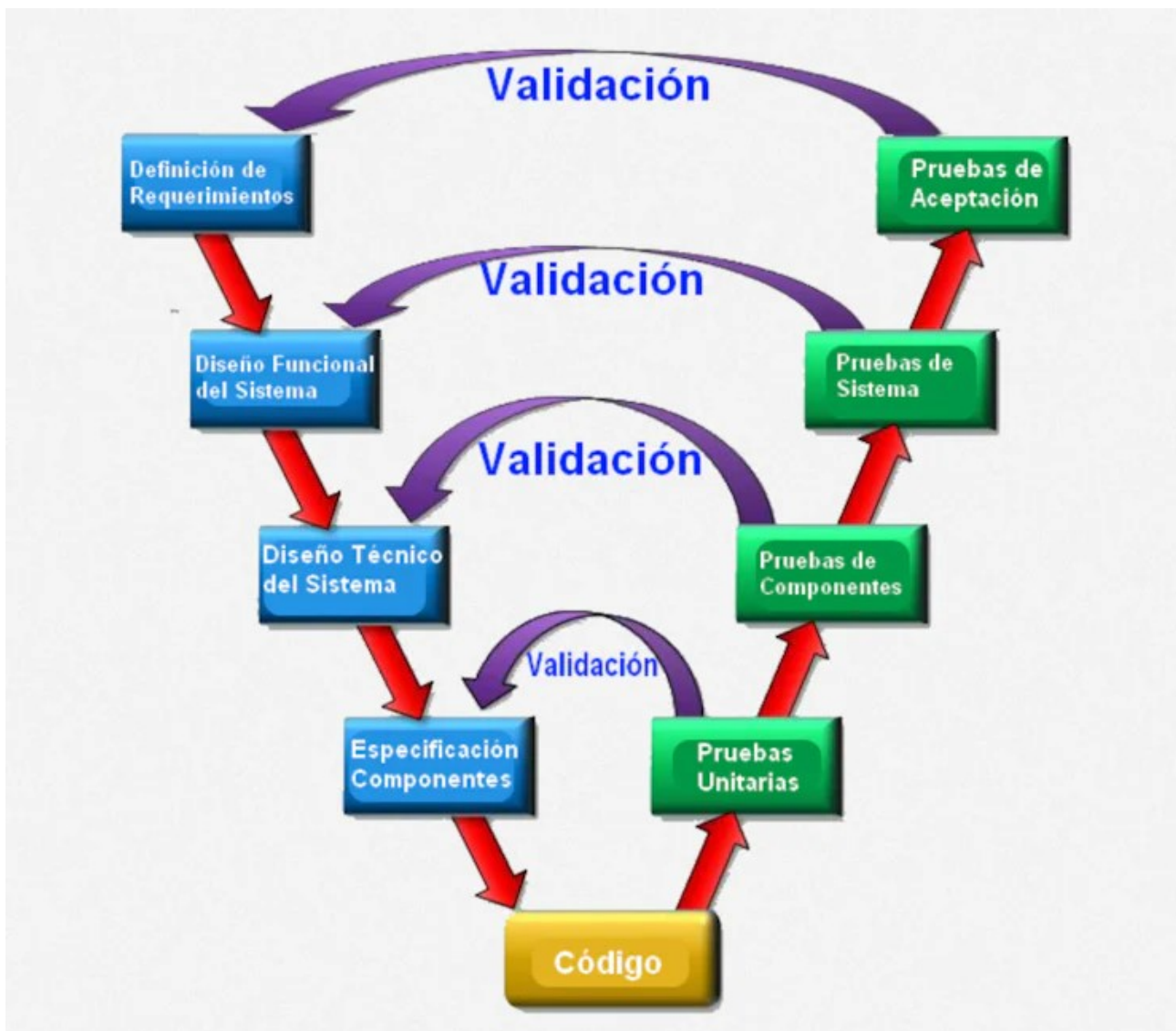


Figura 1.- Modelo en V

## 2.1 Prueba unitarias

Se comprueban que cada una de las piezas o unidades más pequeñas del software que se está desarrollando funcione correctamente. Estas pruebas se aplican de manera individual y son las primeras que deben realizarse durante todo el proceso de desarrollo.

Como lo más probable es que el paradigma de programación que se está empleando es el orientado a objeto, se considera la unidad más pequeña del software la clase. En el caso de la programación estructurada y modular, la unidad de software más pequeña es el módulo.

En el caso de probar las clases, la prueba unitarias conlleva realizar también pruebas a nivel de método de la clase, es decir, para cada método no trivial habrá que comprobar si su comportamiento es el adecuado. Por tanto, para probar correctamente una clase entera, es necesario realizar pruebas en cada uno de sus métodos.

Por tanto, es necesario aislar esas unidades de software para verificar su comportamiento. Al trabajar con unidades tan pequeñas es posible probar el proyecto por partes, sin necesidad de que esté terminado.

## 2.2 Prueba de integración

Estas pruebas se realizan después de las anteriores. Una vez una clase está probada, es el momento de comprobar si actúa bien cuando se ejecuta con otras clases. Se asume que una clase que funciona bien de forma aislada no tiene por qué funcionar bien cuando opera con otras. El objetivo es comprobar si las clases de las que consta el software funcionan correctamente cuando operan entre ellas con el objetivo de realizar las tareas encomendadas a la aplicación.

Hay dos estrategias en los sistemas orientados a objetos:

1. Prueba basada en hilo → Un hilo consta del conjunto de clases necesarias para responder a una entrada o evento del sistema. Cada hilo se prueba de manera individual.
2. Prueba basada en uso → Se comienza probando las clases independientes, que son aquellas que usan muy pocas clases o ninguna. Después de probar estas, se examina la siguiente capa de clases, llamadas clases dependientes, que son utilizadas por las primeras. Esta secuencia de pruebas para las capas de clases dependientes continúa hasta que se construye todo el sistema.

## 2.3 Prueba de sistema

Este tipo de prueba consiste en el proceso de prueba de un sistema integrado de hardware y software para comprobar si cumple con los requisitos especificados, los cuales son:

- ✓ Cumplimiento de todos los requisitos funcionales, considerando el producto software final al completo, en un entorno de sistema.
- ✓ El funcionamiento y rendimiento de las interfaces hardware, software, de usuario y de operador.
- ✓ Adecuación de la documentación de usuario.
- ✓ Ejecución y rendimiento en condiciones límite y de sobrecarga.

Es posible distinguir varios tipos de pruebas del sistema:

- ✓ Pruebas de recuperación → Se fuerza un fallo software y se verifica que el proceso de recuperación se realiza adecuadamente. Este tipo de prueba debe aplicarse a sistemas tolerantes a fallos y en los que se requiere una recuperación rápida.
- ✓ Pruebas de seguridad → Para algunos sistemas, por su naturaleza, es importante protegerlos ante intrusos cuyo fin es dañar el sistema u obtener información confidencial. Las pruebas de seguridad intentan verificar que los mecanismos de

protección del sistema lo protegerán contra accesos ilegales. Se trata de que la persona que lleva a cabo la prueba intente acceder al sistema, para conseguir contraseñas, provocar errores del sistema, etc. El objetivo es comprobar el comportamiento del sistema ante estas situaciones y determinar en qué medida la dificultad que supone el intento de penetración en el sistema no compensa los beneficios que pueda conseguir la persona que lo intenta penetrar.

- ✓ **Pruebas de esfuerzo** → Se trata de forzar al sistema a operar en situaciones extremas para comprobar su comportamiento en estas circunstancias anormales que ocurren excepcionalmente. Se diseñan casos de prueba que requieran memoria máxima y otros recursos hardware, peticiones excesivas de búsqueda de datos residentes en disco, etcétera.
- ✓ **Pruebas de rendimiento** → Están diseñadas para probar el rendimiento del sistema. Se realizan en aquellos sistemas para los que no es suficiente el cumplimiento de los requisitos funcionales, sino que lo hagan con ciertos requisitos de rendimiento, como por ejemplo que se dé respuesta a un determinado evento en un tiempo limitado.
- ✓ **Pruebas de despliegue** → También llamadas pruebas de configuración, estas se realizan cuando es necesario que la aplicación se ejecute en diversas plataformas y sistemas operativos. Consisten en probar el funcionamiento del sistema en las diferentes plataformas en las que se debe poder utilizar. Por ejemplo, en el caso del desarrollo de una aplicación web, puede ser necesario probar su funcionamiento en diferentes navegadores y en distintos servidores web, incluyendo todas las posibles combinaciones de plataforma, servidor web y navegador.

## 2.4 Prueba de validación

También conocida como prueba de aceptación. Consiste en determinar si el software desarrollado es considerado válido por parte del cliente o usuario final y si está preparado para su implantación en el entorno en el que se va a utilizar.

Se considera que el software es válido si cumple una serie de criterios de validación previamente establecidos y que forman parte de la especificación de requisitos del software (ERS) realizada en la fase de análisis del ciclo de vida.

En este tipo de pruebas, no lo realizan en exclusiva el equipo de pruebas. Aquellos usuarios que lo vayan a utilizar también participan activamente ejecutando las pruebas, siendo asesorados por los primeros. Esta es la fase final del proceso software, en la que el cliente da su consentimiento para su explotación.

## 2.5 Prueba de regresión

En ocasiones, el desarrollador decide modificar alguna funcionalidad en el sistema después de haber sido sometido a pruebas. La prueba de regresión tienen como objetivo comprobar que los cambios en un componente del software no provocan errores, carencia de funcionalidad o divergencias respecto al comportamiento esperado tanto en en otros elementos no modificados como al software final. No tendría sentido introducir una nueva

función si esto va a tener consecuencias negativas sobre el resto.

Se evalúa el correcto funcionamiento del software desarrollado frente a evoluciones o cambios funcionales. El propósito de éstas es asegurar que los casos de prueba que ya habían sido probados y no detectaron errores permanezcan así.

## 3 Diseño de casos de prueba

---

Una técnica de diseño de casos de prueba es una forma de generar casos de prueba para probar el software. Dependiendo del enfoque que se adopte se puede aplicar uno de los dos siguientes tipos de técnicas de diseño de casos de prueba:

1. Pruebas de caja blanca o pruebas estructurales → Examina cada detalle del código fuente del módulo. Se deben recorrer todos los posibles caminos que puede generar el flujo del programa.
2. Pruebas de caja negra o pruebas funcionales → Se realizan pruebas sin conocer los detalles internos del módulo. Simplemente se proporcionan una serie de entradas de datos y se comprueban las salidas proporcionadas por el módulo. El objetivo es validar los requisitos funcionales.

Es habitual y recomendable utilizar los dos tipos de diseño de casos de prueba ya que no son excluyentes.

### 3.1 Pruebas de la caja blanca o estructurales

El objetivo de las pruebas de caja blanca es crear casos de prueba que permitan revisar detalles internos del software. Así, de esta técnica se derivan casos de prueba que:

1. Garantizan que se recorren al menos una vez todos los posibles flujos de control de un módulo.
2. Se revisan todas las expresiones lógicas de las estructuras de control en sus vertientes verdadera y falsa.
3. Se ejecutan todos los bucles en sus límites y dentro de sus límites operativos.
4. Se revisan estructuras de datos internas para garantizar su validez.

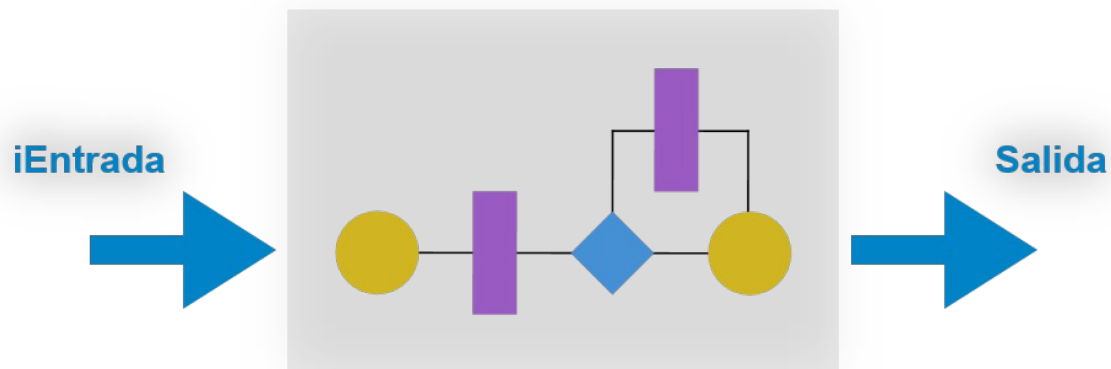


Figura 2.- Pruebas de la caja blanca

Existen diferentes técnicas de diseño de casos de prueba de caja blanca, que vamos a ver a continuación.

### 3.1.1 Prueba del camino básico

También conocida como prueba de ruta básica y fue propuesta por Thomas J. McCabe. Esta técnica genera una métrica de la complejidad del módulo que se va a probar denominada complejidad ciclomática de McCabe. Consiste en un número entero que indica el número de caminos independientes que hay en el código fuente y, por tanto, el número de casos de prueba que hay que ejecutar para garantizar que se recorren todos esos caminos independientes al menos una vez. Esto significa que todas las sentencias del código fuente serían ejecutadas cuando se ejecutaran todos los casos de prueba.

Para calcular esta métrica y, en consecuencia, generar los casos de prueba, es necesario representar el código fuente que se va a probar en un grafo de flujo. Este grafo está compuesto por nodos, que representan las sentencias del código fuente, y aristas, las cuales unen los nodos indicando la secuencia de ejecución de las sentencias.

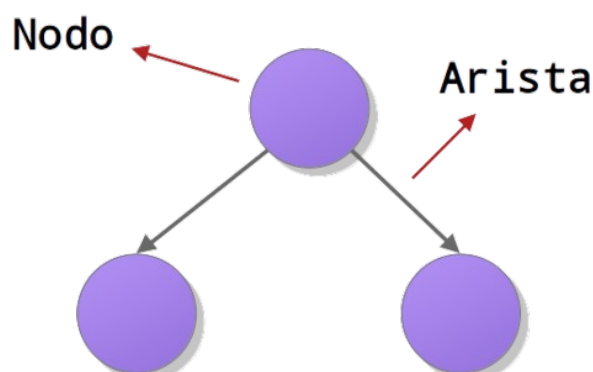


Figura 3.- Nodos y aristas del grafo

Para crear el grafo de flujo tendremos que seguir los siguientes pasos:

1. Se asigna un número único a cada sentencia del código y a cada condición. No obstante, si se dispone de varias sentencias en secuencia, es posible asignar un número único a todas ellas y estas serán tratadas como un nodo único en el grafo. Cada uno de estos elementos (sentencia o grupo de sentencias en secuencia o condición) constituirá un nodo del grafo de flujo, que se representará mediante un círculo en cuyo interior aparecerá el número asignado.
2. Se unen los nodos mediante flechas llamadas aristas que representan el flujo de control u orden de ejecución de las sentencias. Se deben representar las estructuras básicas de programación tal y como se muestra en la figura siguiente.

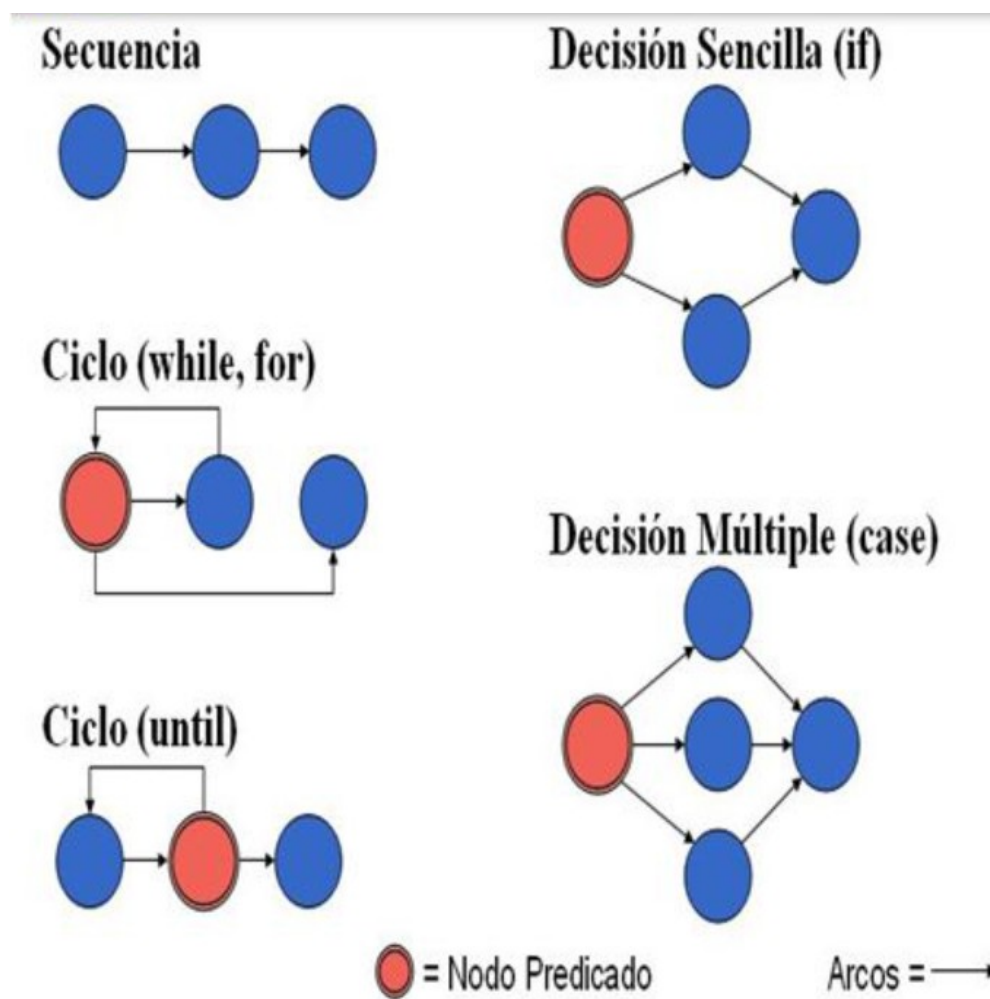


Figura 4.- Estructuras básicas de programación

Una vez creado el grafo de flujo, se puede calcular la complejidad ciclomática de McCabe de cualquiera de las tres formas siguientes:

- ✓ Número de regiones del grafo, considerando una región como una zona del grafo cerrada. No obstante, también se debe considerar como región el grafo completo.

- ✓ De acuerdo con la fórmula:

$$V(G) = A - N + 2$$

donde A es el número de aristas del grafo, y N, el número de nodos.

- ✓ Según la ecuación:

$$V(G) = NP + 1$$

donde NP es el número de nodos predicados del grafo de flujo, es decir, el número de nodos que contienen condiciones.

El valor  $V(G)$  indica el número de caminos independientes del grafo y, por tanto, el número de casos de prueba que hay que generar para garantizar que se han recorrido todas instrucciones y condiciones del código.

El último paso de este método es generar casos de prueba que recorran cada uno de los caminos, para lo que se deben elegir los datos de entrada que fueren cada uno de esos caminos. Puede ocurrir que alguno de los caminos no pueda recorrerse de manera independiente, sino como parte de otro camino.

Para cada caso de prueba, se deben comparar los resultados obtenidos con los esperados. La no coincidencia indicará la existencia de algún defecto y, en consecuencia, se debe llevar a cabo un proceso de depuración.

Veamos un ejemplo. Supongamos el siguiente programa escrito en Python que lee números por teclado hasta que se introduce el 0. Posteriormente visualiza la media aritmética de los números positivos y negativos.

```
cont_pos = 0
cont_neg = 0
suma_pos = 0
suma_neg = 0

numero = int(input("Introduce un número (0 fin): "))
while numero != 0:

    if numero > 0:
        cont_pos = cont_pos + 1
        suma_pos = suma_pos + numero
    else:
        cont_neg = cont_neg + 1
        suma_neg = suma_neg + numero
    # End if

    numero = int(input("Introduce un número (0 fin): "))
# End while

if cont_pos > 0:
    print(f"Se han introducido {cont_pos} números")
    print("positivos")
    print(f"La media aritmética de los números")
```

```

        (" positivos es {suma_pos/cont_pos}")
# End if

if cont_neg > 0:
    print(f"Se han introducido {cont_neg} números")
    ("negativos")
    print(f"La media aritmética de los números")
    ("negativos es {suma_neg/cont_neg}")
# End if

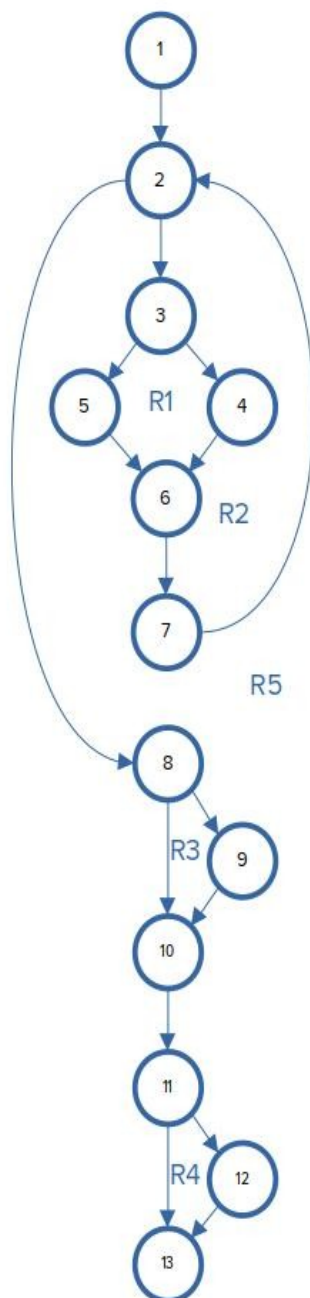
```

Creamos los nodos del grafo

| Nodo | Sentencias                                                                                                                                                                    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <pre> cont_pos = 0 cont_neg = 0 suma_pos = 0 suma_neg = 0  numero = int(input("Introduce un número (0 fin): ")) </pre>                                                        |
| 2    | <pre> while numero != 0: </pre>                                                                                                                                               |
| 3    | <pre>     if numero &gt; 0: </pre>                                                                                                                                            |
| 4    | <pre>         cont_pos = cont_pos + 1         suma_pos = suma_pos + numero </pre>                                                                                             |
|      | <pre>     else: </pre>                                                                                                                                                        |
| 5    | <pre>         cont_neg = cont_neg + 1         suma_neg = suma_neg + numero </pre>                                                                                             |
| 6    | <pre>     # End if     numero = int(input("Introduce un número (0 fin): ")) </pre>                                                                                            |
| 7    | <pre> # End while </pre>                                                                                                                                                      |
| 8    | <pre> if cont_pos &gt; 0: </pre>                                                                                                                                              |
| 9    | <pre>     print(f"Se han introducido {cont_pos} números")     ("positivos")     print(f"La media aritmética de los números")     (" positivos es {suma_pos/cont_pos}") </pre> |
| 10   | <pre> # End if </pre>                                                                                                                                                         |
| 11   | <pre> if cont_neg &gt; 0: </pre>                                                                                                                                              |

|    |                                                                                                                                                                  |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12 | <pre> print(f"Se han introducido {cont_neg} números")   ("negativos") print(f"La media aritmética de los números")   ("negativos es {suma_neg/cont_neg}") </pre> |
| 13 | <pre> # End if </pre>                                                                                                                                            |

Si representamos el grafo tendremos lo siguiente:



Una vez que se tiene el grafo de flujo, ya se puede calcular la complejidad ciclomática,  $V(G)$ , de las tres formas que se ha indicado antes:

- ✓  $V(G) = \text{número de regiones} = 5$
- ✓  $V(G) = A - N + 2 = 17 - 14 + 2 = 5$
- ✓  $V(G) = NP + 1 = 4 + 1 = 5$

Como  $V(G)$  toma el valor 5, se deduce que hay cinco caminos independientes en el código examinado y, por tanto, el número de casos de prueba que hay que crear para cubrir todas las instrucciones y condiciones del programa es 5. La siguiente tarea es encontrar esos cinco caminos independientes. Para ello, hay que tener en cuenta que cada camino independiente incluye una ruta nueva en relación con los anteriores:

- ✓ Camino 1 → Nodos 1, 2, 8, 10, 11, 13
- ✓ Camino 2 → Nodos 1, 2, 3, 4, 6, 7, 2, 8, 10, 11, 13
- ✓ Camino 3 → Nodos 1, 2, 3, 4, 6, 7, 2, 8, 9, 10, 11, 13
- ✓ Camino 4 → Nodos 1, 2, 3, 5, 6, 7, 2, 8, 10, 11, 13
- ✓ Camino 5 → Nodos 1, 2, 3, 4, 6, 7, 2, 3, 5, 6, 7, 2, 8, 9, 10, 11, 12, 13

Ahora tenemos que generar casos de prueba que recorran los caminos anteriores.

Para probar el camino 1, se debe introducir como primer número el 0 con el fin de que no se entre en el bucle ni una sola vez y se pase directamente del nodo 2 al nodo 9. En este caso, el programa no debe mostrar nada, ya que no se ha introducido ningún número ni positivo, ni negativo.

El camino 2 no puede probarse, ya que al introducir un número positivo pasará por el nodo 4 (cálculo de la media de los números positivos), y luego tiene que pasar por el nodo 9, que no está incluido. Por tanto este camino se prueba dentro del siguiente.

Para probar el camino 3, hay que suministrar al programa, en primer lugar, un número positivo (para pasar por el nodo 4), por ejemplo el 9 y un 0 a continuación para salir del bucle. El programa deberá indicar que la media de los números positivos es 9. Al probar el camino 3, ya se prueba el camino 2 como parte de él.

El camino 4 tampoco puede probarse, ya que si se introduce un número negativo tiene que pasar por el nodo 5 (cálculo de la media de los números negativos), y luego tendrá que pasar obligatoriamente por el nodo 12, que no está incluido. Este camino se probará dentro del siguiente.

Para probar el camino 5, hay que introducir un número negativo, por ejemplo, -9, otro positivo, por ejemplo 11, y después un 0 para salir del bucle. En este caso mostrará la media de los números negativos -9 y positivos 11.

En cada caso de prueba, comprobaremos que el resultado coincide con el esperado. Si no fuera así sería indicativo de la existencia de algún defecto, por lo que hay que detectarlo y corregirlo mediante un proceso de depuración.

La complejidad ciclomática de McCabe es una métrica de la complejidad del módulo

probado. Este valor permite determinar la cantidad de casos de pruebas que serán necesarios para evaluar la totalidad de los posibles flujos del código. Esto asegura haber probado el código en su totalidad, y reduce la inestabilidad del software.

Existen diferentes estándares para evaluar un programa dependiendo esta métrica, y los valores pueden variar ligeramente según el caso o lenguaje de programación. A continuación se muestra una de ellas:

| V(G)  | Descripción                                                                   |
|-------|-------------------------------------------------------------------------------|
| 1-10  | Código bien estructurado<br>Capacidad de testeo alta<br>Costo y esfuerzo bajo |
| 10-20 | Código complejo<br>Capacidad de testeo media<br>Costo y esfuerzo medio        |
| 20-40 | Código muy complejo<br>Capacidad de testeo baja                               |
| >40   | No testeable<br>Costo y esfuerzo alto                                         |

Existen diferentes métodos y técnicas que pueden utilizarse durante y después del desarrollo para disminuir la complejidad ciclomática de un código y, de esta manera, reducir el costo de mantenimiento, prueba y modificación de un programa.

Durante el desarrollo de un software es una buena práctica obtener esta métrica para cada módulo que se esté creando. Si este número es más alto de lo recomendado (10 o 15 dependiendo del autor) es necesario dividir el módulo en módulos más pequeños.

Al escribir funciones es necesario tener en cuenta *el principio de responsabilidad única*. Cada función del software debe tener solo una responsabilidad, y todo el código que no forme parte del núcleo de dicha responsabilidad deberá ser extraído en una función propia. Esta práctica aumentará también la usabilidad del código, ya que cada función se encargará de hacer una tarea específica y evitará el código duplicado.

Finalmente es importante evitar lo más posible el uso excesivo de estructuras de control y parámetros opcionales en las funciones. Estos elementos crearán más bifurcaciones dentro del código y, por lo tanto, aumentarán su complejidad ciclomática.

### 3.1.2 Prueba de bucles

Los bucles o estructuras repetitivas constituyen una estructura de control fundamental en todo lenguaje de programación. La prueba de bucles se centra exclusivamente en este tipo de estructuras. Los casos de prueba que se deben generar para cada tipo de bucle son

los siguientes:

- ✓ Bucles simples → Si el número máximo de iteraciones por el bucle es  $n$ , se deben probar los siguientes casos:
  - Saltarse completamente el bucle
  - Realizar una iteración por el bucle.
  - Realizar dos iteraciones por el bucle.
  - Realizar  $m$  iteraciones por el bucle, siendo  $m < n$ .
  - Realizar  $n-1$ ,  $n$  y  $n+1$  iteraciones por el bucle.
- ✓ Bucles anidados → Se deben llevar a cabo los siguientes pasos:
  - Comenzar por el bucle más interno, estableciendo valores mínimos para todos los demás bucles.
  - Realizar pruebas de bucle simple para el bucle más interno, manteniendo los bucles exteriores en sus valores mínimos.
  - Trabajar de dentro hacia fuera y probar el siguiente bucle manteniendo los otros bucles exteriores en valores mínimos y los otros bucles interiores en valores típicos.
  - Continuar hasta que se hayan probado todos los bucles.
- ✓ Bucles concatenados → Si los bucles son independientes unos de otros, se pueden probar empleando la estrategia indicada para los bucles simples. Si los bucles concatenados son dependientes porque el segundo bucle usa el contador del primer bucle como valor inicial para el segundo, se recomienda emplear el enfoque aplicado a los bucles anidados.

### 3.1.3 Prueba de flujo de datos

Esta técnica de diseño de casos de prueba selecciona caminos de un programa de acuerdo con las ubicaciones de las definiciones y usos de variables en dicho programa.

Aquí es necesario considerar que, en una sentencia, se define una variable si se le asigna valor, mientras que se puede afirmar que esa variable se usa si se accede en lectura a su valor de algún modo. Así, por ejemplo, en una condición, siempre se usan una o varias variables.

Si se asigna el número **1** a una sentencia, el conjunto **DEF** para esa sentencia **1** esta formado por el conjunto de variables que se definen en esa sentencia. Su conjunto **USO** estará formado por el conjunto de variables cuyo valor se lee en esa sentencia. Es decir:

$$\begin{aligned} \text{DEF}(1) &= \{ X \mid \text{sentencia 1 contiene una definición de } X \} \\ \text{USO}(1) &= \{ X \mid \text{sentencia 1 contiene una lectura de } X \} \end{aligned}$$

La definición de una variable **X** en la sentencia **1** está viva en la sentencia **1'** si hay

camino entre las instrucciones **1** e **1'** que no contiene ninguna otra definición de **X**.

Mediante esta técnica, se encuentran las cadenas **DU** (definición uso) para una variable. Estas cadenas tienen la forma **[X, 1, 1']**, donde **X** es una variable, **1** e **1'** son sentencias tal que **X** está **DEF (1)** y en **USO (1')**, y existe algún camino entre **1** y **1'** que no incluye ninguna otra definición de **X**.

Esta técnica requiere seguir los siguientes pasos:

1. Asignar un número a cada sentencia y condición del programa.
2. Calcular el conjunto **DEF** y **USO** para cada variable que se usa en ese programa.
3. Encontrar las cadenas **DU** para todas las variables.
4. Generar el número mínimo de casos de prueba que incluyan caminos para todas las cadenas **DU**.
5. Generar datos de entrada que fueren cada uno de los caminos encontrados en el paso anterior.

Vamos a utilizar el mismo ejemplo visto con las pruebas del camino básico. El primer paso ya está hecho ya que en el ejemplo anterior tenemos asignado un número a cada sentencia y condición.

Seguidamente, se calculan las cadenas **DU** solo para la variable **numero**. Para ello, se definen los conjuntos **DEF** y **USO** para esta variable. Como se puede observar, la variable **numero** se le asigna valor en las sentencias 1 y 6, mientras que se usa en las que tienen asignados los números 2, 3, 4 y 5. Esto se indica del siguiente modo:

```
DEF (1) = {numero}
USO (2) = {numero}
USO (3) = {numero}
USO (4) = {numero}
USO (5) = {numero}
DEF (6) = {numero}
```

El siguiente paso es buscar las cadenas **DU** para esta variable, que son las siguientes numeradas:

```
DU[numero,1,2] (1)
DU[numero,1,3] (2)
DU[numero,1,4] (3)
DU[numero,1,5] (4)
DU[numero,6,2] (5)
DU[numero,6,3] (6)
DU[numero,6,4] (7)
DU[numero,6,5] (8)
```

A continuación, se debe encontrar el número mínimo de caminos que incluyan todas las cadenas **DU**. Estos caminos son:

- ✓ Camino 1 → Nodos 1-2-3-4-6-7-2-3-4-6-7-2-3-5-6-7-2-8-9-10-11-12-13. Este camino cubre

las cadenas DU: (1), (2), (3), (5), (6), (7) y (8).

- ✓ Camino 2 → Nodos 1-2-3-5-6-7-2-8-10-11-12-13. Este camino cubre la cadena DU (4)

En este caso, con dos caminos es suficiente para cubrir todas las cadenas DU encontradas. En la primera iteración del bucle del camino 1, se cubren las cadenas DU (1), (2) y (3); en la segunda iteración, se cubren las cadenas DU (5), (6) y (7) y, en la tercera iteración, se cubre la cadena DU (8). Pero, en el camino 1, no es posible cubrir la cadena DU con el número (4), es decir, **DU[numero, 1, 5]** porque por el nodo 1 solo se pasa una vez. En las iteraciones, no se puede pasar por los nodos 4 y 5 a la vez y, para volver a uno de los nodos 4 o 5, hay que pasar por el nodo 6, que contiene otra definición de la variable **numero**. Por este motivo, hay que establecer otro camino para cubrir esta cadena DU.

Ya solo quedaría generar casos de prueba con los datos de entrada para recorrer cada uno de estos dos caminos. Para el camino 1, habría que introducir en el programa dos números positivos, luego uno negativo y un cero para finalizar. Por ejemplo, si se suministra un 4, un 8 y un -9, el programa nos debe indicar que la media de los números positivos es 6 y la de los negativos, -9. Para forzar el camino 2, habría que introducir en el programa un número negativo y un cero para finalizar. Si, por ejemplo, se proporciona como entrada el número -10 y luego un 0, el programa debe indicar que la media de los números negativos es -10.

## 3.2 Pruebas de la caja negra o funcionales

El objetivo de las pruebas de caja negra es crear casos de prueba que permitan revisar todos los requisitos funcionales de un programa. A tal efecto, existen diferentes técnicas de diseño de casos de prueba de caja negra que se exponen a continuación.

### 3.2.1 Particiones o clases de equivalencia

Por medio de esta técnica, se identifican un conjunto de clases de equivalencia o tipos de valores que se deben asignar a los datos de entrada. El primer paso para identificar las clases de equivalencia es determinar las condiciones de entrada del programa. Una condición de entrada puede ser:

- ✓ Un valor específico.
- ✓ Un rango de valores permitido, por ejemplo, entre 23 y 47.
- ✓ Un conjunto de valores permitidos, por ejemplo, 23, 37 y 89.
- ✓ Una condición lógica o booleana, por ejemplo, está habilitado o no está deshabilitado.

Hay una serie de reglas que ayudan a identificar las clases de equivalencia en función del tipo de la condición de entrada:

- ✓ Si la condición de entrada es un valor específico, se debe crear una clase válida (con ese valor) y dos no válidas (con dos valores distintos).
- ✓ Si la condición de entrada consiste en un rango de valores permitidos (por ejemplo,

un número entre 23 y 47), se debe crear una clase válida ( $23 < \text{número} < 47$ ) y dos no válidas ( $\text{número} < 23$  y  $\text{número} > 47$ ).

- ✓ Si la condición de entrada consiste en un conjunto de valores permitidos y se sabe que el programa trata de manera diferente cada uno de ellos (por ejemplo, el idioma es inglés, francés o italiano), se identifica una clase válida por cada valor (en este caso, tres valores) y una clase no válida (por ejemplo, alemán).
- ✓ Si la condición de entrada consiste en una condición lógica (por ejemplo, la persona debe tener nacionalidad española); se identifica una clase de equivalencia válida (es española) y otra no válida (no es española).
- ✓ Si se sabe que algunos elementos de una clase no se tratan de igual forma que el resto de sus elementos, debe dividirse dicha clase de equivalencia en varias clases menores.

Para la creación de los casos de prueba, una vez conocidas estas reglas, se llevan a cabo los siguientes tres pasos:

1. Asignar un número único a cada clase de equivalencia.
2. Crear el número mínimo de casos de prueba que incluyan todas las clases válidas.
3. Crear un caso de prueba por cada clase no válida. El caso de prueba debe incluir una clase no válida, y todas las demás clases deben ser válidas.

Veamos un ejemplo. Supongamos el siguiente programa:

*En una aplicación hay un módulo que permite al usuario realizar una búsqueda de vehículos de alquiler que necesita los siguientes datos:*

- ✓ *La matrícula de un vehículo, que es una cadena de caracteres con 4 números y tres letras;*
- ✓ *El precio diario, que es un número positivo entre 50 y 200 €.*
- ✓ *El motor tiene solo puede ser: gasolina, diésel, híbrido o eléctrico.*
- ✓ *Hay que indicar que el pago tiene que ser con tarjeta bancaria.*

En la tabla siguiente se muestran las condiciones de entrada y, por cada una de ellas, las clases válidas y las no válidas, numerándose cada una de ellas.

| Condición de entrada | Clase válida                                        | Clase no válida                                                                                                                                                      |
|----------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Matrícula            | Cadena de 7 caracteres con 4 números y 3 letras (1) | Cadena < 7 caracteres (2)<br>Cadena > 7 caracteres (3)<br>Alguno de los 4 primeros caracteres no es número (4)<br>Alguno de los 3 últimos caracteres no es letra (5) |

|               |                                                                |                                                                 |
|---------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Precio diario | 50 <= precio <= 200 (6)                                        | Precio < 50 (7)<br>Precio > 200 (8)<br>No es número (9)         |
| Motor         | Gasolina (10)<br>diésel (11)<br>híbrido (12)<br>eléctrico (13) | Un valor diferente a gasolina, diésel, híbrido o eléctrico (14) |
| Pago          | Tarjeta bancaria (15)                                          | No tarjeta bancaria (16)                                        |

En las siguientes tablas, se muestran los casos de prueba válidos y los no válidos. Por cada caso de prueba, se indica, en la última columna, las clases de equivalencia incluidas. Recuerda que, para el caso de las clases válidas, se debe crear el número mínimo de casos de prueba que incluyan todas las clases válidas.

| Matrícula | Precio diario | Motor     | Pago             | Clases incluidas |
|-----------|---------------|-----------|------------------|------------------|
| 1234MNP   | 100           | Gasolina  | Tarjeta bancaria | 1, 6, 10, 15     |
| 1234MNP   | 100           | Diésel    | Tarjeta bancaria | 1, 6, 11, 15     |
| 1234MNP   | 100           | Híbrido   | Tarjeta bancaria | 1, 6, 12, 15     |
| 1234MNP   | 100           | Eléctrico | Tarjeta bancaria | 1, 6, 13, 15     |

Para las clases no válidas, hay que crear un caso de prueba por cada una de ellas. Como se puede observar en la siguiente tabla, al haber 9 clases de equivalencia no válidas, se deben crear 9 casos de prueba.

| Matrícula | Precio diario | Motor     | Pago             | Clases incluidas |
|-----------|---------------|-----------|------------------|------------------|
| 1234M     | 100           | Gasolina  | Tarjeta bancaria | 2, 6, 10, 15     |
| 1234MNPP  | 100           | Diésel    | Tarjeta bancaria | 3, 6, 11, 15     |
| W234MNP   | 100           | Híbrido   | Tarjeta bancaria | 4, 6, 12, 15     |
| 1234MN8   | 100           | Eléctrico | Tarjeta bancaria | 5, 6, 13, 15     |
| 1234MNP   | 40            | Gasolina  | Tarjeta bancaria | 1, 7, 10, 15     |
| 1234MNP   | 210           | Gasolina  | Tarjeta bancaria | 1, 8, 10, 15     |
| 1234MNP   | Caro          | Gasolina  | Tarjeta bancaria | 1, 9, 10, 15     |
| 1234MNP   | 100           | A pedales | Tarjeta bancaria | 1, 6, 14, 15     |

|         |     |          |         |              |
|---------|-----|----------|---------|--------------|
| 1234MNP | 100 | Gasolina | Contado | 1, 6, 13, 16 |
|---------|-----|----------|---------|--------------|

### 3.2.2 Análisis de valores límite

Se trata de una técnica de diseño de casos de prueba complementaria a la de clases de equivalencia. Las diferencias que presenta con relación a esta son:

- ✓ En vez de seleccionar cualquier elemento dentro de una clase de equivalencia, se selecciona aquel o aquellos que se hallan justo en los límites de la clase.
- ✓ En vez de centrarse únicamente en las condiciones de entrada, en esta técnica, los casos de prueba se generan teniendo en cuenta también el dominio de salida.

Las reglas que se siguen para generar los casos de prueba son las siguientes:

1. Si una condición de entrada especifica un rango de valores, por ejemplo, entre -5 y 10, se deben generar casos válidos para los extremos del rango (-5 y 10) y casos no válidos justo más allá de los extremos (-5,01 y 10,01, en caso de que se admitan dos decimales).
2. Si una condición de entrada especifica un número de valores (entre 1 y 100), hay que generar casos de prueba para los valores mínimo y máximo (1 y 100), uno menos que el mínimo (0) y uno más que el máximo (101).
3. Se emplea la regla 1 para cada condición de salida que especifique un rango de valores. Si, por ejemplo, el salario mínimo es 1.200,00 € y el máximo es 3.600,00 €, se deben generar casos de prueba que den como resultado salarios de 1.200,00 €, 3.600,00 €, 1.199,99 € y 3.600,01 €.
4. Se usa la regla 2 para cada condición de salida que especifique un número de valores. Si, por ejemplo, el programa especifica que se puede hacer uso de uno a tres descuentos, habrá que generar casos de prueba para lograr cero, uno, tres y cuatro descuentos.
5. Si la entrada o salida de un programa es un conjunto ordenado (por ejemplo, una tabla o un archivo secuencial), los casos de prueba se deben centrar en el primer y último elemento.

Por ejemplo, en el ejemplo del epígrafe anterior el *precio* tenía que estar entre 50 y 200. Dado que esta condición de entrada especifica un número de valores (entre 50 y 200), hay que generar clases de equivalencia para los valores mínimo y máximo (50 y 200), uno menos que el mínimo (49) y uno más que el máximo (201), como se muestra en la siguiente tabla, en la que se ha asignado a las clases de equivalencia una numeración que sigue la del ejemplo anterior.

| Condición de entrada | Clases válidas                        | Clases no válidas                     |
|----------------------|---------------------------------------|---------------------------------------|
| Precio               | precio = 50 (17)<br>precio = 200 (18) | Precio = 49 (19)<br>precio = 201 (20) |

Una vez generada estas clases de equivalencia adicionales, se trata de generar los casos de prueba que abarquen estas clases. Para ello, se debe tener en cuenta que hay que generar un caso de prueba válido que abarque todas las clases válidas. Como, en este caso, para el *precio*, hay dos clases válidas, se deben generar dos casos de prueba válidos como en la siguiente tabla. A la edad, se deben añadir las otras condiciones de entrada que se incluyeron en el ejemplo anterior.

| Matrícula | Precio diario | Motor    | Pago             | Clases incluidas |
|-----------|---------------|----------|------------------|------------------|
| 1234MNP   | 50            | Gasolina | Tarjeta bancaria | 1, 17, 10, 15    |
| 1234MNP   | 200           | Gasolina | Tarjeta bancaria | 1, 18, 10, 15    |

Hay que generar, además, un caso de prueba nuevo por cada clase no válida. Como se han añadido dos nuevas clases no válidas, se generará un caso de prueba no válido por cada una de ellas, como se muestra en la siguiente tabla.

| Matrícula | Precio diario | Motor    | Pago             | Clases incluidas |
|-----------|---------------|----------|------------------|------------------|
| 1234MNP   | 49            | Gasolina | Tarjeta bancaria | 1, 20, 10, 15    |
| 1234MNP   | 201           | Gasolina | Tarjeta bancaria | 1, 21, 10, 15    |

### 3.2.3 Conjetura de errores

Esta técnica consiste en generar una lista de errores que suelen cometer quienes desarrollan aplicaciones y de las situaciones que suelen dar lugar a dichos errores, para luego generar un conjunto de casos de prueba basándose en esta lista. Se trata de errores que se cometen comúnmente y no relacionados con aspectos funcionales.

Se muestran a continuación una serie de ejemplos típicos que reflejan el empleo de esta técnica:

- ✓ El valor 0 tanto en la entrada como en la salida es una situación propensa a que se cometan errores.
- ✓ En situaciones en las que se introduce un número variable de valores, como por ejemplo una lista, conviene considerar los siguientes casos: no introducir ningún valor, introducir un solo valor y un conjunto de valores en el que todos son iguales.
- ✓ El usuario de la aplicación puede introducir datos erróneos, por ejemplo, que en un campo numérico introduzca algún carácter no numérico.
- ✓ La persona encargada de la programación pueda haber interpretado incorrectamente algo en la especificación.

## 3.3 El proceso de pruebas del software

El proceso de prueba de una aplicación completa se debe ejecutar de acuerdo con los siguientes pasos:

- ✓ Se comienza realizando la prueba de unidad de cada clase. Se puede comenzar con la técnica de análisis de valores límite, añadir clases válidas y/o no válidas no contempladas con la técnica de clases de equivalencia y finalizar con la técnica de conjetura de errores.
- ✓ Si no se ha alcanzado la cobertura deseada (por ejemplo, el recorrido de todos los posibles caminos a lo largo del código fuente), se deben añadir los casos necesarios mediante técnicas de pruebas de caja blanca (es decir, la prueba del camino básico, la de bucles y la de flujo de datos).
- ✓ Se continúa con las pruebas de integración, aunque también se pueden combinar estas con pruebas de unidad. Las pruebas de integración se deben centrar en la comprobación de los mecanismos de interacción entre las diferentes clases y se deben usar preferentemente técnicas de caja blanca. Se debe ir incrementando progresivamente el alcance de estas pruebas hasta abarcar el software completo.
- ✓ La prueba del sistema debe emplear casos de prueba de caja negra para comprobar el cumplimiento de los objetivos requeridos por el sistema.
- ✓ En la prueba de validación, deben tomar parte los usuarios finales y se pueden reutilizar casos de prueba del sistema. La finalización con éxito de estas pruebas supone la aceptación final del software por parte del usuario.

## 4 Documentación de las pruebas

---

La documentación de las pruebas es conveniente para una buena organización de estas y para asegurar su reutilización. Se deben documentar tanto el diseño de las pruebas como el resultado o ejecución de las pruebas.

### 4.1 Diseño de los casos de prueba

En cuanto a la documentación del diseño de las pruebas, es conveniente crear los siguientes documentos:

- ✓ **Plan de pruebas** → Es un documento con la planificación general de las pruebas para la aplicación que se está creando. Se debe indicar el enfoque general de las pruebas, los recursos requeridos, las actividades de pruebas que se van a llevar a cabo, los elementos que se deben probar, el personal responsable y los riesgos asociados.
- ✓ **Especificación del diseño de las pruebas** → Es un informe que detalla el plan de pruebas porque en él se indica cada uno de los elementos concretos que se van a probar y los procedimientos de prueba. A partir de este documento se generan los siguientes:
  - Especificaciones de casos de prueba → Por cada prueba que se va a realizar, se deben indicar los datos de entrada que se van a suministrar, los resultados esperados y las dependencias entre casos de prueba.
  - Especificaciones de procedimientos de prueba → Especifica los pasos para la

ejecución de uno o de un conjunto de casos de prueba. En este documento, se deben indicar la lista de casos de prueba correspondientes al procedimiento, los requisitos para la ejecución de los casos de prueba (entorno, personal, etc.) y los pasos del procedimiento.

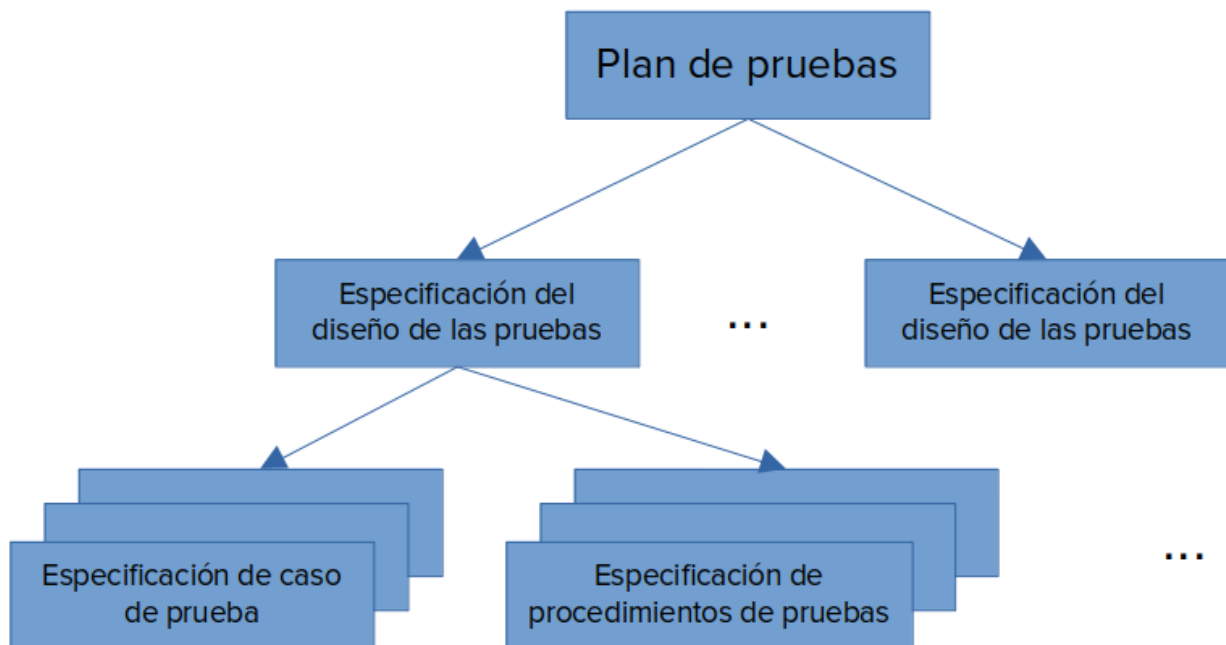


Figura 5.- El plan de pruebas

El plan de pruebas es un documento único para una aplicación con indicaciones generales acerca de las pruebas. Este se desglosa, por cada elemento que se va a probar, en varias especificaciones del diseño de las pruebas. Cada uno de estos documentos contiene además varias especificaciones de casos de prueba y varias especificaciones de procedimientos de pruebas.

## 4.2 Ejecución de las pruebas

En cuanto a la documentación de la ejecución de las pruebas, esta es importante para la eficacia en la detección y corrección de defectos y para dejar constancia de los resultados de las pruebas. La ejecución de las pruebas toma como entrada las especificaciones de los casos de prueba que se van a usar y las especificaciones de los procedimientos de prueba, y debe generar dos tipos de documentos:

1. **Histórico de pruebas** → Registra todos los hechos relevantes ocurridos durante la ejecución: elementos probados, entorno de pruebas, resultados obtenidos, fecha y hora y referencia al informe de incidentes, si es el caso.
2. **Informe de incidentes** → Registra cada incidente ocurrido durante la prueba que requiera una posterior investigación.

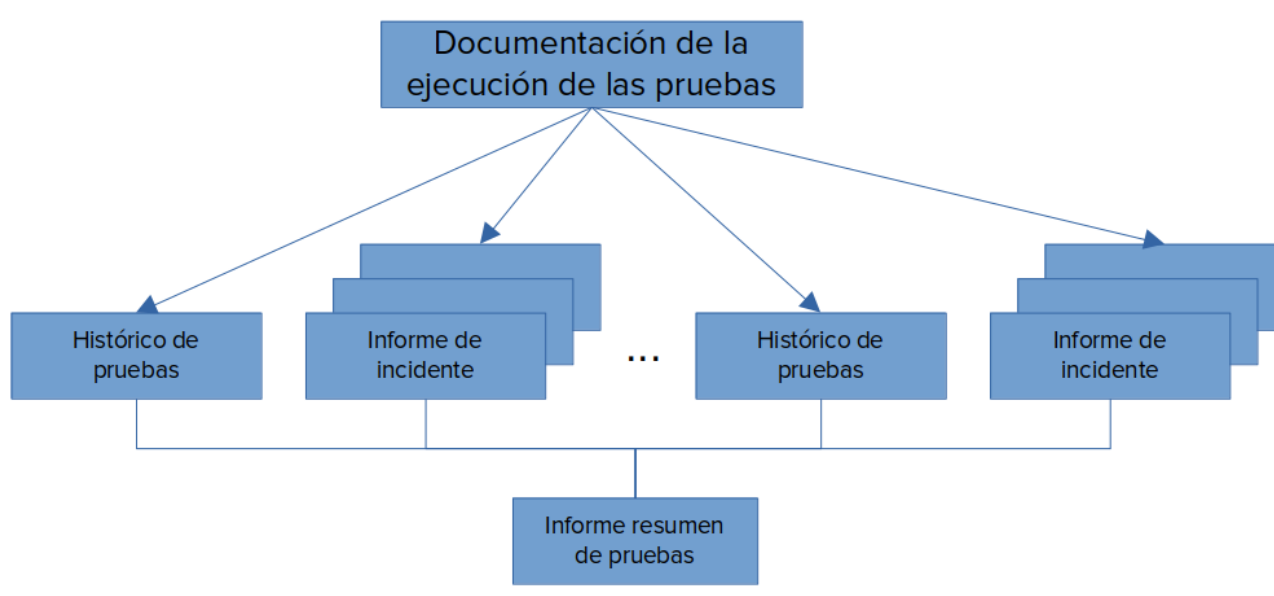


Figura 6.- Documentación de ejecución de las pruebas

Se parte de una especificación del diseño de las pruebas y, por cada ejecución de estas, a partir una serie de casos de pruebas y sus procedimientos, se genera un histórico de pruebas con todos hechos relevantes ocurridos durante esas pruebas y un informe por cada incidente ocurrido. A partir de aquí se crea un informe resumen de pruebas que contiene:

- ✓ Un resumen de la evaluación de los elementos probados.
- ✓ Variaciones del software realizadas como resultado de las pruebas.
- ✓ Valoración de la cobertura de la prueba.
- ✓ Resumen de los resultados obtenidos y de las actividades de prueba.

La valoración de los procesos de prueba es importante porque puede ayudar en proyectos posteriores. Por ello, es conveniente realizar un **análisis causal**, cuyo objetivo es proporcionar información sobre la naturaleza de los defectos encontrados. Por cada defecto encontrado, se debería registrar la siguiente información: cuándo se cometió, quién lo cometió, qué se hizo mal, cómo se podría haber evitado, por qué no se detectó antes, cómo se podría haber detectado antes y cómo se encontró el error.

El objetivo del análisis causal es formar al personal sobre los errores cometidos para que se puedan prevenir en el futuro. La información generada se puede emplear para predecir los fallos futuros del software.

## 5 Calidad del software

El desarrollo de software es una disciplina complicada y muchos de los productos que se desarrollan tienen calidad insuficiente, además de no cumplir las estimaciones de tiempo y recursos para el desarrollo de los mismos. Todos los métodos, herramientas y procedimientos que constituyen la Ingeniería del Software van orientados a un único fin:

producir software de calidad.

Se define la calidad del software como

*«la concordancia con los requerimientos funcionales y de rendimiento explícitamente establecidos, con los estándares de desarrollo documentados y con las características implícitas que se esperan de todo software desarrollado profesionalmente»*

Se puede decir que los requisitos del software son la base de las medidas de calidad y que la falta de concordancia con los requisitos es una falta de calidad. Se deben evitar los requisitos implícitos, es decir, aquellos no incluidos en la ERS, porque ello podría dar lugar a problemas a la hora de considerar el software válido o no.

A la hora de definir la calidad del software se debe diferenciar entre la calidad del producto software y la calidad del proceso de desarrollo. No obstante, las metas que se establezcan para la calidad del producto van a determinar las metas a establecer para la calidad del proceso de desarrollo, ya que la calidad del producto va a estar en función de la calidad del proceso de desarrollo. Sin un buen proceso de desarrollo es casi imposible obtener un buen producto.

Las metodologías de desarrollo definen un conjunto de tareas para el desarrollo software y como se llevan a cabo. Si no se sigue ninguna metodología siempre habrá falta de calidad. Todas las metodologías y herramientas tienen un único fin, producir software de alta calidad.

## 5.1 Medidas de calidad del software

Una métrica es la evaluación del grado en el cual un producto o proceso posee un atributo determinado (extensión, cantidad, dimensiones, capacidad o tamaño). Las métricas pueden ser de tres tipos:

- ✓ Métricas del producto → Describen las características del producto y hacen referencia a la calidad del software desarrollado.
- ✓ Métricas de proceso → Se refiere a las características del método utilizado en el desarrollo del software. Estas características pueden utilizarse para mejorar las actividades de desarrollo y mantenimiento del software.
- ✓ Métricas del proyecto → Estas métricas describen las características del proyecto y su ejecución. Se puede usar para medir la eficiencia de un equipo de proyecto o cualquier herramienta que utilicen los miembros del equipo.

### 5.1.1 Métricas del producto

Las más habituales son:

- ✓ Tiempo medio entre fallos → Es el tiempo que transcurre, por término medio, entre un fallo del software y el siguiente. Esta métrica habitualmente se utiliza en sistemas con seguridad crítica.

- ✓ Densidad de defectos → Mide los defectos relativos al tamaño del software expresados como líneas de código o puntos de función. El número de defectos que son introducidos en el código por cierta cantidad de líneas de código (LOC) que son programadas en ese producto. Generalmente, esta medida define estadísticamente que por cada mil líneas de código que se escriben, estaremos esperando que el programa tenga un número específico de errores en él.
- ✓ Problemas del cliente → Mide los problemas que el cliente encuentra cuando utiliza el software. Contiene la perspectiva del cliente desde el contexto del uso del software.
- ✓ Satisfacción del cliente → Mide el cumplimiento de los requisitos del software desde el punto de vista del cliente. Frecuentemente se usan datos proporcionados por el cliente a través de una escala categórica de cinco puntos: muy satisfecho, satisfecho, neutro, insatisfecho, muy insatisfecho.

### 5.1.2 Métricas del proceso

Son medidas cuantitativas que permiten a los ingenieros de software obtener una visión de la eficacia del proceso de software. Entre las más habituales están:

- ✓ Tamaño del software en miles de líneas de código.
- ✓ La complejidad.
- ✓ Características de diseño.
- ✓ Rendimiento

### 5.1.3 Métricas del proyecto

Las utiliza el gestor del proyecto para comprobar el progreso del proyecto. Los datos de proyectos pasados se utilizan para definir varias métricas, como el tiempo y el coste. Esto permite realizar estimaciones como base para el desarrollo de un nuevo software, intentando reducir los costes de desarrollo y la mejora la calidad del proyecto. A medida que la calidad mejora, el número de errores, tiempo y coste, se reducen. Entre las más habituales están:

- ✓ Esfuerzo necesario en número de desarrolladores o dotación de personal para las fases del ciclo de vida.
- ✓ Coste.
- ✓ Tiempo de desarrollo y calendario.
- ✓ Productividad.

## 5.2 Normas y certificaciones

Los siguientes son estándares ampliamente reconocidos y utilizados en la industria en relación a la calidad del software:

- ✓ ISO 9001. Norma para la implementación de un método o Sistema de Gestión de la

Calidad (SGC), supone la acreditación de la capacidad para satisfacer los requisitos de calidad. Aporta así una serie de requisitos genéricos (no circunscritos al software) y aplicables a cualquier organización.

- ✓ ISO 10005:2018. Ofrece una guía para gestionar un plan de calidad todo el ciclo de vida.
- ✓ ISO IEC 25000. Supone una familia de normas también conocida como SQuaRE (*Software product Quality Requirements and Evaluation*), define un marco de referencia para la calidad del producto de software. Evalúa un software en 8 áreas, incluyendo Adecuación funcional, fiabilidad, usabilidad, eficiencia, compatibilidad, seguridad, mantenibilidad y portabilidad.
- ✓ ISO 33000 Calidad de los procesos de desarrollo de software. Se enfoca en la evaluación de la calidad de los procesos de desarrollo de software, y también se lo conoce como SPICE (*Software Process Improvement and Capability Determination*). Así, busca conocer la evolución en el tiempo sobre los procesos de desarrollo, hacer un seguimiento respecto a la competencia y determinar posibles estrategias de mejora.
- ✓ CMMI (*Capability Maturity Model Integration*). Proporciona un marco de referencia para evaluar y mejorar la madurez de los procesos en el desarrollo de software. Se centra en áreas como la gestión de proyectos, de la calidad, la configuración, o riesgos, entre otros.
- ✓ ISO 12207 Modelos de Ciclos de Vida del Software. Proporciona un estándar para observar los procesos de ciclo de vida del software, desde la idea inicial hasta la retirada del software.
- ✓ IEEE 730 – 2002 *Standard for Software Quality Assurance Plans*. Define qué es el software de alta calidad y propone una elaboración de un Plan de Aseguramiento de la calidad de software (SQAP).
- ✓ ISO 5055 *Software Quality Standards*. En este caso, se trata de medir las debilidades críticas de un software, fijándose en los criterios de seguridad, confianza, eficiencia de rendimiento y mantenibilidad.

## 6 Depuración de código

---

Cuando una prueba en el software detecta un error, o el síntoma de un defecto, hay que realizar una depuración del código. La depuración consiste en localizar el defecto en el software y corregirlo.

En ocasiones, la localización del defecto es una tarea sencilla, pero en otras es más complicado. Debido a ello, todos los IDEs incorporan una herramienta de depuración de código.

Una vez detectado el error o defecto, se corrige y se llevan a cabo las modificaciones en el código fuente, lo que puede provocar la introducción de nuevos errores o defectos. En

este caso, es conveniente volver a probar el código mediante pruebas de regresión. Consistiría en volver a ejecutar los casos de prueba que se ejecutaron antes de las modificaciones para asegurarse que dichos cambios no son el origen de nuevos defectos en la aplicación.

Generalmente, el depurador de todos los IDEs presentan un funcionamiento similar y consiste en:

- ✓ Ejecución del código paso a paso.
- ✓ Establecer puntos de ruptura.
- ✓ Suspender la ejecución del programa.
- ✓ Examinar el valor de las variables o expresiones a lo largo de la ejecución.

Vamos a ver la depuración de código fuente con dos IDEs: PyCharm y VS Code.

No hay un procedimiento único para depurar aplicaciones. Dependiendo de los requisitos actuales, es posible que tengamos que usar diferentes acciones en diferente orden. Aquí proporcionamos un criterio general el cual representa los pasos típicos en una depuración. Los detalles de cómo y cuándo usar las características particulares se detallarán más adelante.

1. Definir donde el programa necesita ser interrumpido. Esto habitualmente se hace utilizando puntos de ruptura. Un punto de ruptura es una marca especial que representa los lugares y las condiciones donde el depurador necesita pasar y congelar el estado del programa. Un programa interrumpido por el depurador se dice que está suspendido.

La alternativa a los puntos de ruptura es suspender la ejecución del programa en cualquier momento. Sin embargo, este método impone algunas limitaciones a la funcionalidad del depurador y no permiten mucha precisión respecto al momento de suspensión del programa.

2. Ejecutar el programa en modo depuración. Generalmente, esta opción se encuentra en el menú *Run* del IDE.
3. Después de suspender el programa, utilizamos el depurador para obtener información sobre el estado del programa y como cambia durante la ejecución.

El depurador proporciona información sobre el valor de las variables, el estado actual de los hilos de ejecución, el estado de los objetos, etc. También permite probar el programa en varias condiciones lanzando excepciones (para comprobar como se gestionan) o ejecutar código arbitraria justo en mitad de la ejecución del programa.

Mientras estas herramientas nos permiten examinar el estado del programa en un instante particular, la característica de ejecución paso a paso nos permite controlar la ejecución del programa. Combinando de estas herramientas podremos deducir donde está el error.

## 6.1 PyCharm

Durante una sesión de depuración, se lanza la ejecución del programa con el depurador. El propósito de éste es interferir la ejecución del programa y proporcionar información sobre lo que está pasando entre bastidores. Esto facilita el proceso de detectar y corregir errores en el programa.

### 6.1.1 Puntos de ruptura

Un punto de ruptura (*breakpoint*) es una marca especial que suspende la ejecución del programa en una sentencia concreta. Esto nos permite examinar el estado y comportamiento del programa. Los puntos de ruptura pueden ser simples, los cuales suspenden la ejecución el programa al alcanzar una línea de código, o implicar una lógica más compleja, como por ejemplo comprobando condiciones adicionales, escribiendo mensajes log, etc.

Una vez establecidos, los puntos de ruptura permanecen en el proyecto hasta que los quitemos explícitamente, excepto para los puntos de ruptura temporales.

En PyCharm disponemos de dos tipos de puntos de ruptura:

- ✓ De línea → Suspenden la ejecución del programa cuando se alcanza una línea de código donde ha sido establecido. Este tipo se puede establecer en cualquier línea de código ejecutable.
- ✓ Excepción → Suspenden el programa cuando se lanza una excepción (objeto de la clase Exception o alguna subclase). Se aplican globalmente a las condiciones de la excepción y no requieren referencias a código fuente en particular.

Tenemos dos formas de establecer un punto de ruptura de línea:

- ✓ Hacemos clic en el hueco entre el número de línea de código y la sentencia en esa línea donde queremos establecer el punto de ruptura.
- ✓ Colocar el cursor en la línea y pulsar Ctrl + F8.

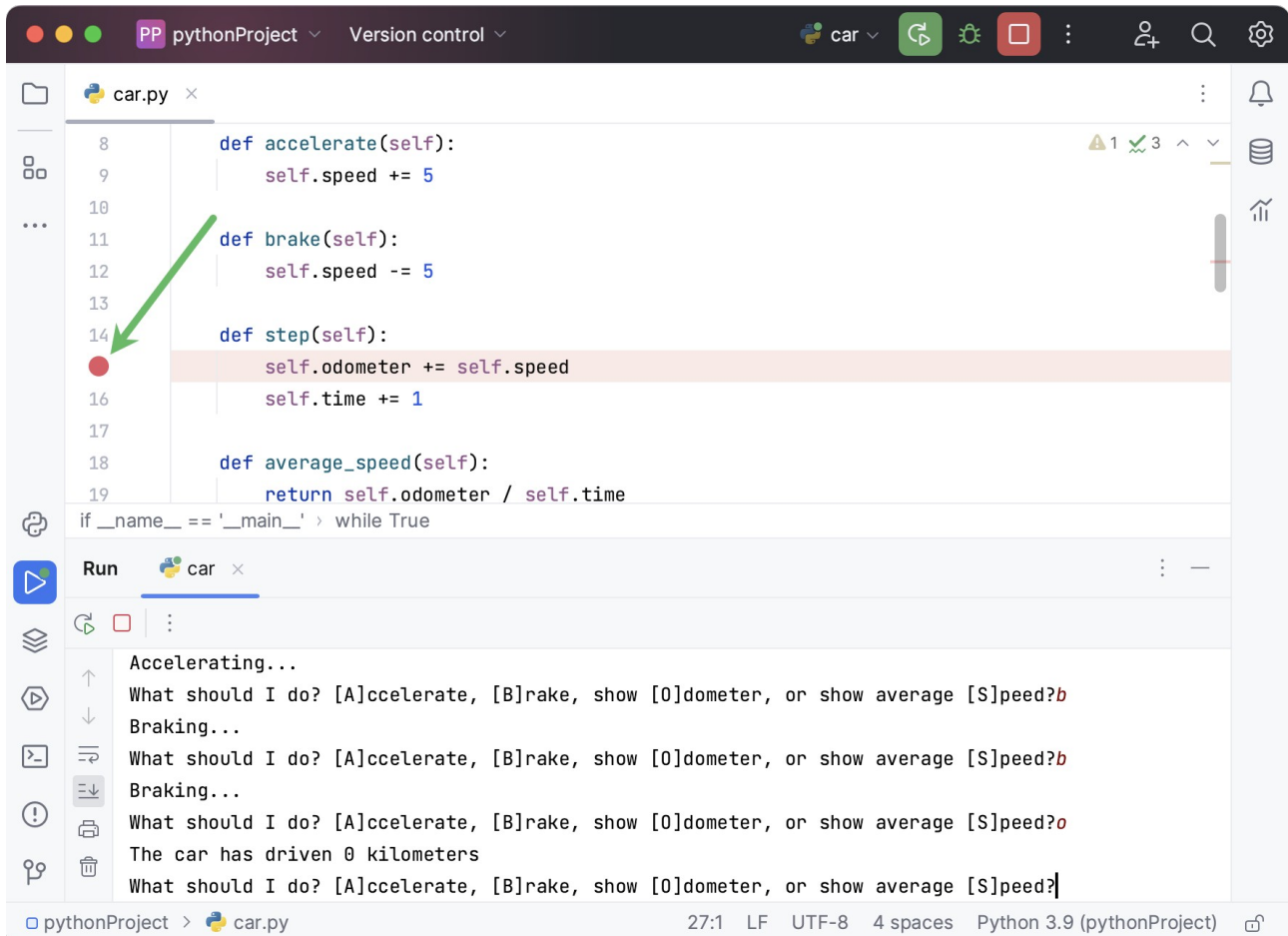


Figura 7.- Establecer punto de ruptura

Si queremos quitar el punto de ruptura solo tenemos que hacer clic sobre el punto rojo. También en la opción de menú *Run → View Breakpoints* seleccionamos el punto de ruptura y hacemos clic en el botón *Remove*.

Si en una sesión de depuración no necesitamos los puntos de ruptura, podemos silenciarlos. Esto permite recuperar la operación normal del programa sin abandonar la sesión de depuración. Después, se pueden volver a activar y continuar con la depuración. Para ello, durante la sesión de depuración hacemos clic en el botón *Mute Breakpoints* en la barra de herramientas del panel *Debug*.

Cuando quitamos un punto de ruptura, su configuración interna se pierde. Para desactivar temporalmente alguno de ellos sin perder sus parámetros, podemos deshabilitarlo de tres formas:

- ✓ Haciendo clic derecho sobre el punto de ruptura y desactivando la opción *Enable*.
- ✓ También, podemos hacer clic directamente en el punto de ruptura con el botón

central del ratón.

- ✓ En el cuadro de diálogo *Breakpoints* (opción de menú *Run → View Breakpoints*) recorreremos la lista de puntos de ruptura y los desactivamos.

Por último, si queremos mover un punto de ruptura lo arrastramos a otra línea del programa.

Los puntos de ruptura disponen de propiedades o atributos que nos permiten ajustar su operación a necesidades específicas. Podemos acceder a ellas en el cuadro de diálogo *Breakpoints*. Son las siguientes:

| Propiedad        | Descripción                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enabled</b>   | Desactivando la casilla <b>Enabled</b> se deshabilita temporalmente el punto de ruptura sin borrarlo. Los puntos de ruptura deshabilitados no se emplean en el proceso de depuración.                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Suspend</b>   | Activando la casilla <b>Suspend</b> la ejecución del programa se para en el punto de ruptura. Suspender la aplicación es útil si necesitamos obtener información o calcular una expresión en un determinado punto sin interrumpirlo. También disponemos de una política de suspensión: <ul style="list-style-type: none"> <li>✓ <b>All</b> → todos los hilos se suspenden.</li> <li>✓ <b>Thread</b> → Solo el hilo que contiene el punto de ruptura se suspende.</li> </ul>                                                                                                       |
| <b>Condition</b> | Se especifica una condición para disparar el punto de ruptura. Esta expresión debe ser válida en la línea donde se establece el punto de ruptura y es evaluada cada vez que se alcanza el punto de ruptura. Si el resultado de la condición es True, las acciones se ejecutan.<br><br>Podemos introducir expresiones de múltiples líneas. Por ejemplo: <pre>if attempted_password == os.environ['USER_PASSWORD']:     session['logged_in'] = True else:     session['logged_in'] = False     error = 'Invalid credentials. Please, try again.'  return session['logged_in']</pre> |
| <b>Log</b>       | Podemos seleccionar crear un mensaje log en la consola de los siguientes tipos: <ul style="list-style-type: none"> <li>✓ <b>Breakpoint hit message</b> → Envían un mensaje a la consola indicando que el punto de ruptura se ha alcanzado.</li> <li>✓ <b>Stack trace</b> → Cuando se alcanza el punto de ruptura, su traza se visualiza en la consola.</li> </ul> <p>Esto es útil si queremos comprobar que flujo de ejecución nos ha llevado a</p>                                                                                                                               |

|                                        |                                                                                                                                                                                                                                         |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        | actual punto de interrupción de la ejecución del programa.                                                                                                                                                                              |
| <b>Evaluate and log</b>                | Activarlo para evaluar una expresión al alcanzar el punto de ruptura y mostrar el resultado en la consola.                                                                                                                              |
| <b>Remove once hit</b>                 | El punto de ruptura de borra cuando se alcanza.                                                                                                                                                                                         |
| <b>Disable until breakpoint is hit</b> | Se emplea para deshabilitar un punto de ruptura que se habilita cuando se alcanza otro. Podemos también indicar si queremos dejarlo deshabilitado de nuevo o dejarlo habilitado una vez se ha alcanzado el punto de ruptura de disparo. |

Dependiendo del tipo y estado del punto de ruptura puede aparecer en el código fuente con un icono diferente. Son los siguientes:

|                         | Line                                                                                | Method                                                                              | Exception                                                                             |
|-------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Regular                 |    |    |    |
| Disabled                |  |  |  |
| Verified                |  |  |                                                                                       |
| Muted                   |  |  |                                                                                       |
| Inactive/dependent      |  |  |                                                                                       |
| Muted disabled          |  |  |                                                                                       |
| Non-suspending          |  |  |                                                                                       |
| Verified non-suspending |  |  |                                                                                       |
| Invalid                 |  |                                                                                     |                                                                                       |

Figura 8.- Iconos de los puntos de ruptura

### 6.1.2 Comenzar la sesión de depuración

Comenzar una sesión de depuración es muy similar a la ejecución del programa en modo normal. El depurador se adjunta a la ejecución, por lo tanto no tenemos que configurar nada específico para comenzar la sesión de depuración.

Cada vez que depuramos un programa, la sesión de depuración se basa en la configuración de ejecución. Así, podemos configurar PyCharm para utilizar parámetros de

entrada y realizar cualquier acción antes de lanzar el programa.

Para iniciar la sesión de depuración podemos hacer algunas de las siguientes acciones:

- ✓ Hacer clic en el botón *Debug* o pulsar Ctrl + F9. Si tuviéramos alguna configuración de ejecución que queramos utilizar, la seleccionamos previamente de la lista de configuraciones de ejecución.
- ✓ En la línea de ejecución del main, hacer clic en el botón  y seleccionar *Debug...*
- ✓ Seleccionar la opción de menú *Run → Debug 'archivo'*. Si tenemos una configuración de ejecución seleccionamos *Run → Debug...* y seleccionamos la configuración de ejecución que necesitamos.

### 6.1.3 Examinar un programa suspendido

Después de iniciar una sesión del depurador, aparece el panel *Debug* y el programa se ejecuta normalmente hasta que ocurra uno de los siguientes:

- ✓ Se alcanza un punto de ruptura.
- ✓ Se suspende el programa manualmente.

Una vez el programa está suspendido, se puede examinar su estado actual, controlar su ejecución posterior y probar varios escenarios en tiempo de ejecución.

#### 6.1.3.1 Examinar marcos

El estado de un programa se representa por marcos. Cuando un programa se suspende, la pila del marco actual se visualiza en la pestaña *Frames* del panel *Debug*.

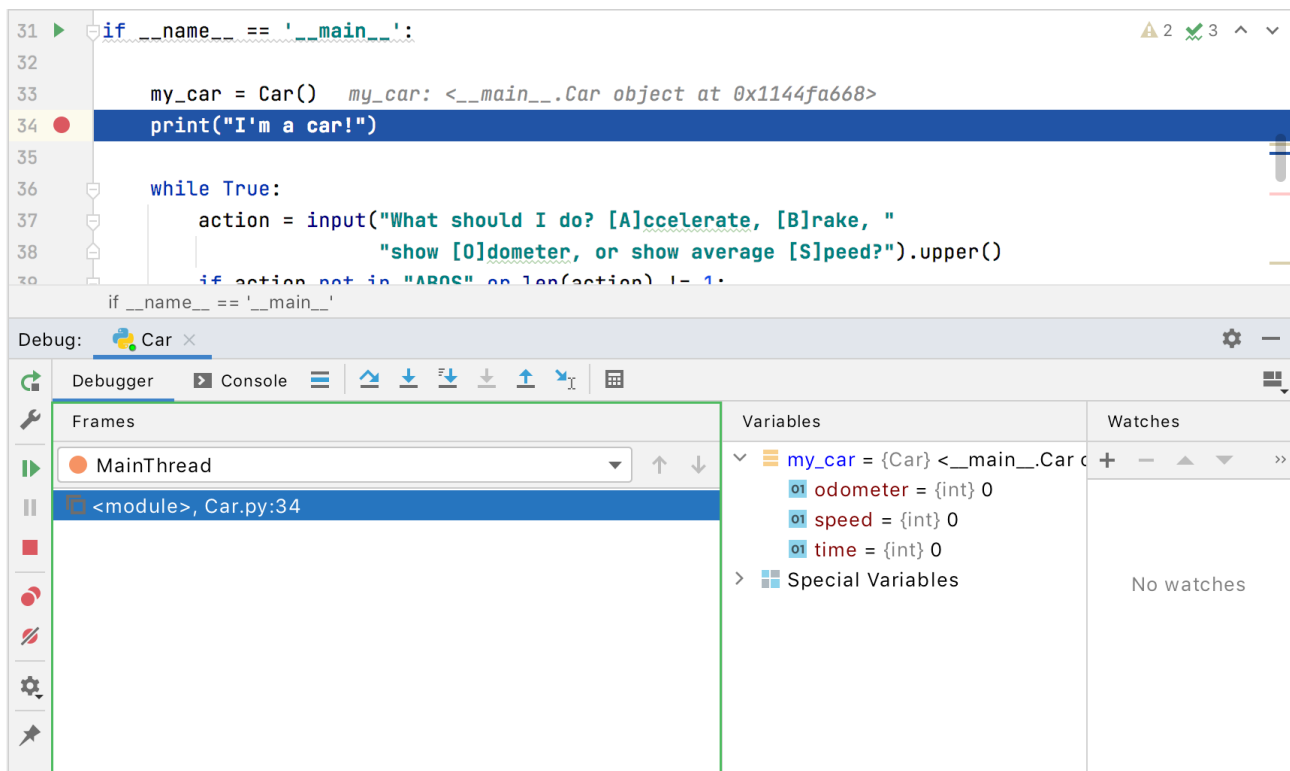


Figura 9.- Panel debug

Un marco se corresponde con la invocación de una función o método activo. Almacena las variables locales de la función o método invocado, sus argumentos y el contexto de código que permite la evaluación de las expresiones.

Cada vez que un método se invoca, un nuevo marco se añade en la cima de la pila. Cuando la ejecución de un método se completa, el marco correspondiente se elimina de la pila al estilo último en entrar, primero en salir.

Cuando seleccionamos un marco, el archivo correspondiente se abre en una pestaña de previsualización, lo que significa que se cerrará cuando naveguemos a otro marco. Para abrir el fichero en una pestaña normal, hacemos doble clic en el marco.

#### 6.1.3.2 Examinar variables

La pestaña variables nos muestra la lista de variables en el marco seleccionado. Examinando su valor nos ayudará a entender porque el programa opera en una determinada forma.

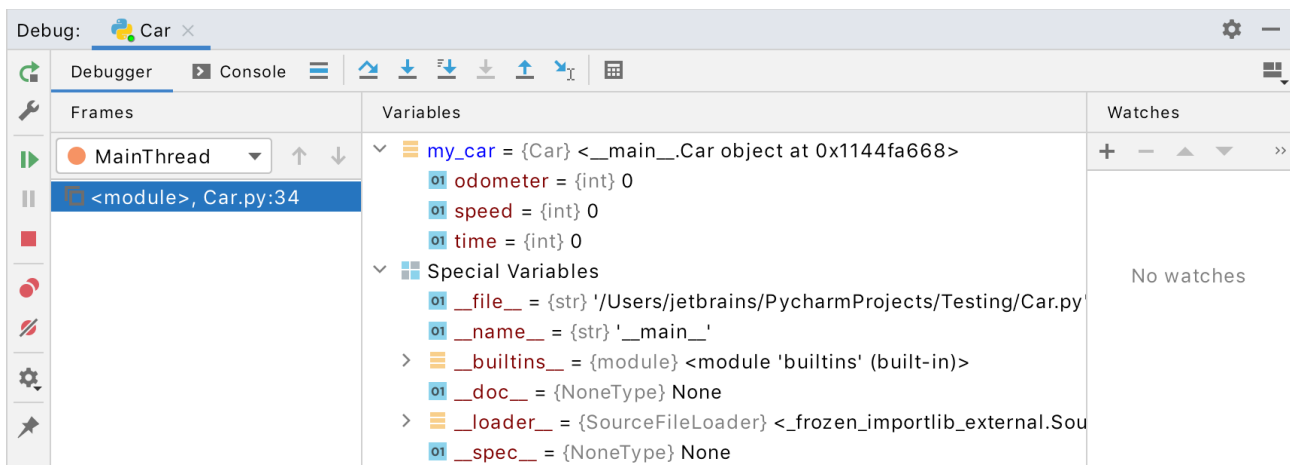


Figura 10.- Pestaña variables

El icono a la izquierda de la variable indica su tipo.

|  |                                    |
|--|------------------------------------|
|  | Local arrays                       |
|  | Local primitive types              |
|  | <u>Watches</u> and auto-variables. |
|  | Local reference variables          |

Figura 11.- Iconos de variables

Cuando examinamos las variables, podemos necesitar copiar el nombre o el valor para pegarlo en otro lugar o compararlo con otra variable. Para copiar el valor de una variable hacemos clic derecho sobre la variable y seleccionamos la opción *Copy Value*. Para copiar su nombre seleccionamos *Copy Name*.

Si necesitamos comparar un valor de variable con algún otro, utilizamos la opción *Compare Value with Clipboard*. Esto es útil, por ejemplo, cuando una variable almacena una cadena muy larga y necesitamos compararla con otra cadena muy larga.

1. Copiamos el contenido que queremos comparar desde un origen, por ejemplo un fichero de texto.
2. En la pestaña *Variables*, clic derecho en la variable que queremos comparar y seleccionamos *Compare Value with Clipboard*.
3. Examinar las diferencias en el *Diff viewer* que se ha abierto.

Si la variable es un array podemos visualizar su valor en una matriz.

1. En la pestaña *Variables* seleccionar una variable de tipo array o *DataFrame*.
2. Clic en el enlace *View as Array* o *View as DataFrame* en la parte derecha.
3. Alternativamente podemos elegir *View as Array* o *View as DataFrame* del menú contextual.
4. Aparece el panel *Data View*.

Si queremos probar como el programa se comporta bajo ciertas condiciones o arreglar su comportamiento actual en tiempo de ejecución, podemos cambiar el valor de las variables.

1. Seleccionamos una variable y pulsamos F2 o clic derecho en la variable y seleccionamos *Set Value*.
2. Introducir el valor para la variable y presionar Enter.

Si necesitamos mirar el código fuente donde la variable o clase se declararon lo podemos hacer desde el panel *Variables*.

1. Para navegar al código donde la variable se ha declarado, hacemos clic derecho en la variable y seleccionamos *Jump to Source*.
2. Para navegar a la declaración de clase del tipo de la variable, clic derecho en la variable y seleccionamos *Jump to Type Source*.

#### 6.1.3.3 Evaluar expresiones

Además de poder ver las variables, PyCharm nos permite evaluar expresiones durante la sesión de depuración para obtener detalles adicionales del estado del programa o probar varios escenarios en tiempo de ejecución. Cuando se evalúan expresiones, debemos tener en cuenta el ámbito y tiempo de vida de la variable. Todas las expresiones se evalúan en el contexto del punto de ejecución actual.

Para evaluar rápidamente una expresión, ponemos el cursor del ratón en la sentencia donde aparece la expresión. Las invocaciones de métodos no pueden evaluarse de esta forma.

1. Apuntar con el ratón a la expresión que queremos evaluar. El resultado de la expresión aparecerá en forma de tooltip.
2. Para ver algunos de los miembros de un objeto hacer pulsar Ctrl + F1.

Si queremos evaluar una expresión en el código que implica una invocación de método, o queremos ser específicos sobre que porción de la expresión se evalúa, utilizamos la opción *Quick Evaluate Expression*.

1. Coloca el cursor en la expresión (para evaluar la expresión coincidente más cercana) o selecciona una porción de ella si queremos ser específicos sobre la parte de la expresión compleja a evaluar.

## 2. Selección la opción *Run → Debugging Actions → Quick Evaluate Expression*.

También podemos evaluar expresiones arbitrarias, lo que es la opción de evaluación de expresiones más flexible. Nos permite evaluar cualquier código dentro del contexto del frame actual. Con ello, podemos evaluar sentencias, invocaciones de funciones, bucles, etc.

Con esta características obtenemos información adicional del estado actual de la ejecución del programa y probamos varios escenarios dentro de la misma sesión de depuración. Esto ahorra mucho tiempo reduciendo el número de sesiones que tenemos que ejecutar.

La forma más rápida de evaluar cualquier expresión es introducirla en el cuadro de texto *expression* del panel *Variables* y pulsar Enter.

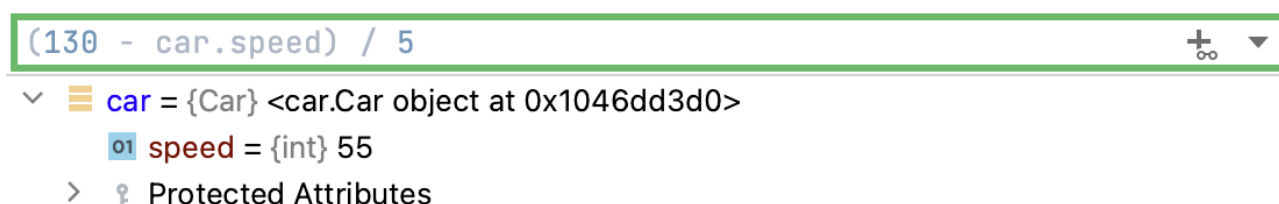


Figura 12.- Evaluación de una expresión

El resultado se visualiza debajo. También podemos añadir la expresión a los *Watches* haciendo clic en el botón *Add to watches* en la parte derecha del cuadro de texto.

Si queremos evaluar un bloque de código largo, podemos utilizar un cuadro de diálogo dedicado.

1. Si queremos comenzar con alguna expresión o variable, actualmente en el editor o en el panel *Variables*, la seleccionamos.
2. Hacemos clic en *Run → Debugging Actions → Evaluate Expression*.
3. En el cuadro de diálogo *Evaluate* modificar la expresión seleccionada o introduce una nueva en el cuadro de texto. Hacer clic en *Expand* para modificar un fragmento de código multilínea.
4. Hacer clic en el botón *Evaluate* (Ctrl+Enter para modo multilínea). El resultado de la expresión aparece en el campo *Result*.

El resultado de la expresión se toma de la sentencia *return*. Cuando no hay sentencia *return*, el resultado se toma de la última línea de código (ni siquiera tiene que ser una expresión, un simple literal también funciona). Cuando no hay una línea válida para tomar un valor, el resultado es indefinido. Si la expresión especificada no puede ser evaluada, el campo *Result* indica la razón.

### 6.1.3.4 Ver valores en línea

PyCharm muestra los valores de las variables a continuación de su uso.

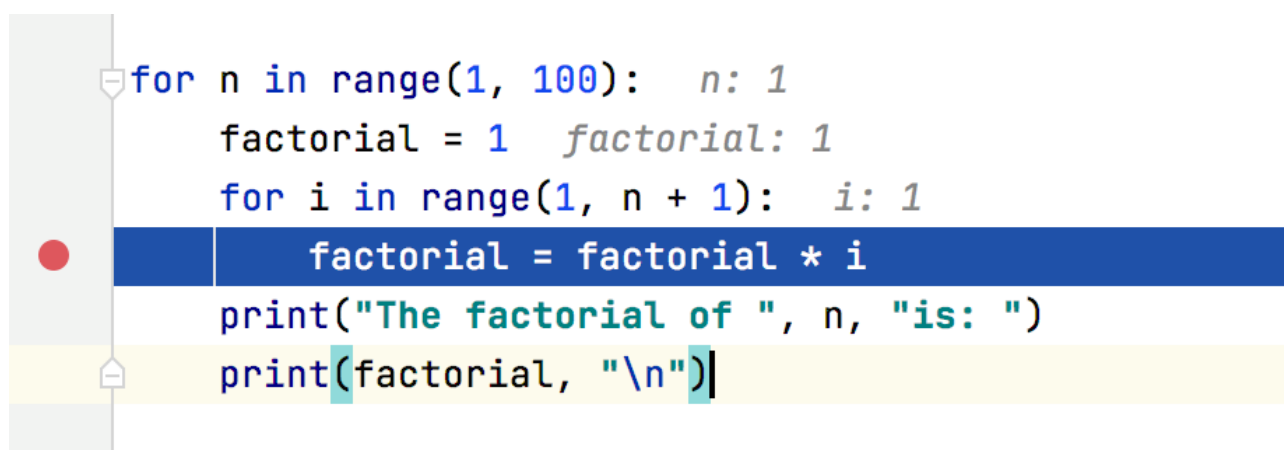


Figura 13.- Valor de las variables en línea

Una vez que el valor de la variable ha cambiado, la vista en línea se actualiza con el nuevo valor y cambia su color.

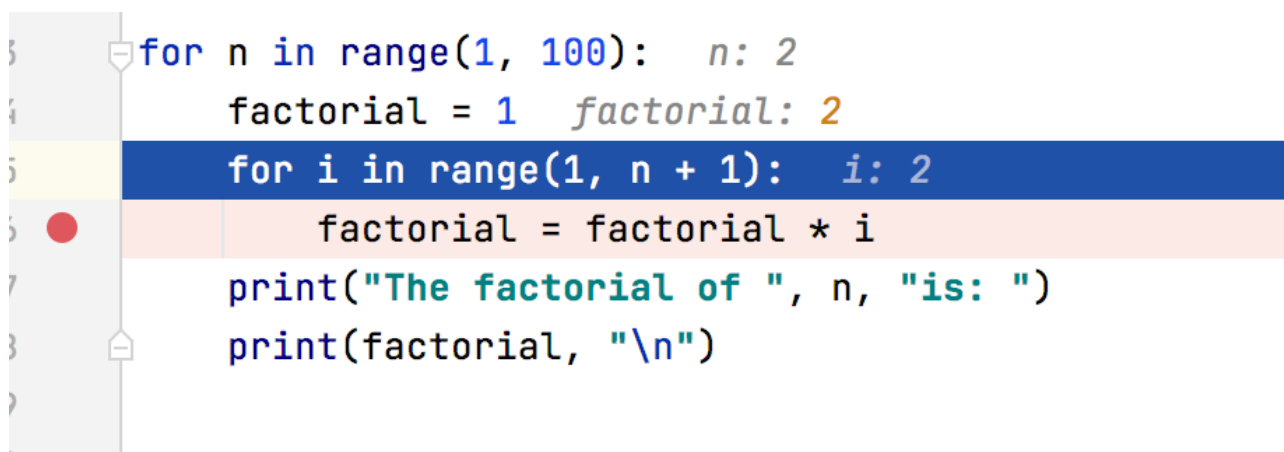


Figura 14.- Cambio de color de variable actualizada

Si una línea contiene una referencia a un objeto, podemos examinar sus propiedades a la derecha del editor. Desde aquí, podemos cambiar el valor de las variables y añadir *watches* en línea.

Si queremos que el resultado de alguna expresión aparezca en una línea particular, podemos establecer una observación en línea para la misma. Las observaciones en línea son persistentes y activas después del reinicio de sesión.

1. Hacemos clic en la variable en línea referida al objeto cuyo miembro queremos seguir.
2. En la ventana, seleccionamos el campo y hacemos clic en *Add as Inline watch*.
3. Ajustamos la observación si se necesita. Podemos utilizar cualquier expresión válida como observación.

Para borrar una observación en línea, hacemos clic en la x que aparece junto a ella al introducir el ratón.

### 6.1.3.5 Observaciones

Si queremos mantener un seguimiento de alguna variable o el resultado de una expresión compleja, podemos establecer una observación para esta variable o expresión. Esto es útil cuando necesitamos añadir algo que normalmente no se visualiza en la lista de variables, o para no tener que expandir el árbol de alguna variable de instancia.

Las observaciones son evaluadas en el contexto del marco seleccionado. Las observaciones no pueden evaluarse si están fuera de contexto o si falla la traducción. En este caso, la evaluación se marca con el icono de error.

Para añadir una observación hacemos lo siguiente:

1. Hacer clic en el botón *New Watch* en la pestaña *Watches*.
2. Introducir la variable o expresión a ser evaluada. En expresiones, podemos evaluar invocaciones de métodos, declarar variables, etc, tanto como se pueda en el contexto local.

Después de introducir la variable/expresión a las observaciones, permanecerá y se evaluará en cada paso, proporcionando el resultado en el contexto actual.

Para editar una observación hacemos clic derecho sobre la misma y seleccionamos *Edit*. Para eliminar una observación simple, hacemos clic derecho sobre ella y seleccionamos *Remove Watch*. Si queremos borrarlas todas, hacemos clic derecho en cualquier lugar del panel de *Variables/Watches* y seleccionamos *Remove All Watches*.

Las observaciones nos permite las mismas acciones que las variables. Por ejemplo, podemos verlas en un cuadro de diálogo dedicado o usarlas para navegar por el código fuente. Las observaciones son parte del proyecto. Esto significa que podemos parar y reejecutar la sesión de depuración sin riesgo de perderlas.

### 6.1.3.6 Regresar al punto de ejecución

Examinar el estado del programa implica navegar por el código y frecuentemente necesitaremos regresar al lugar donde el programa fue suspendido. Para ello seleccionamos la opción de menú *Run → Debugging Actions → Show Execution Point*. El punto de ejecución actual se indica con una línea azul de fondo. El código en esta línea todavía no ha sido ejecutado.

### 6.1.4 Ejecución paso a paso

La ejecución paso a paso es el proceso de controlar la ejecución de cada sentencia del programa. PyCharm proporciona un conjunto de acciones paso a paso las cuales se emplean en función de la estrategia de depuración que sigamos. Por ejemplo, si necesitamos ir directamente a la siguiente línea o entrar en los métodos invocados.

Los botones de ejecución paso a paso se localizan en la barra de herramientas del panel *Debug*.

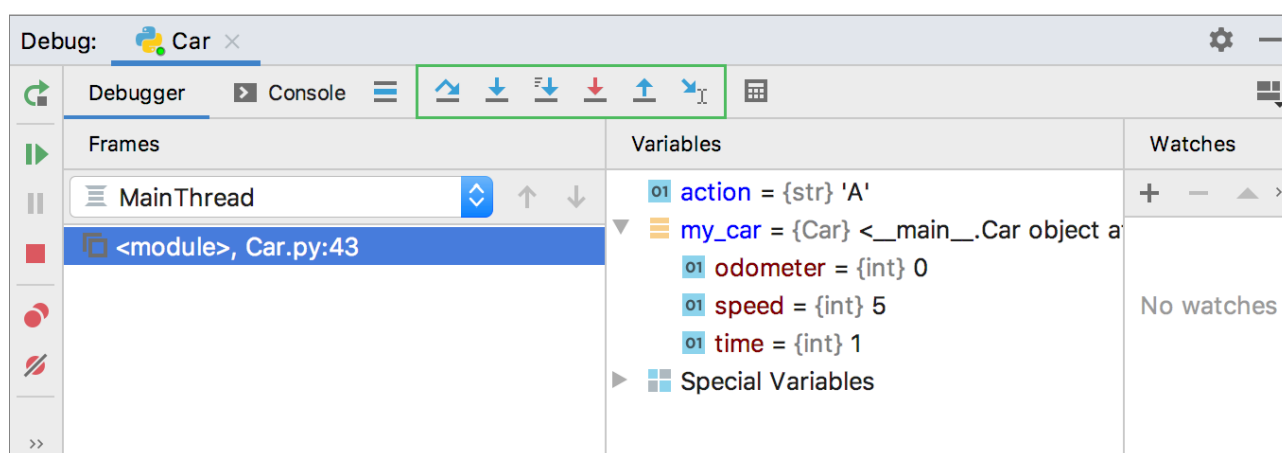


Figura 15.- Ejecución paso a paso

#### 6.1.4.1 Step over

Ejecuta la línea de código actual y luego va a la siguiente línea incluso si la línea resaltada tiene una invocación de método. La implementación de los métodos se salta y nos movemos directamente a la siguiente línea de la invocación del método.

Para ejecutar una línea de código pulsamos F8. Si hay puntos de ruptura dentro de los métodos que nos saltamos, el depurador parará en ellos. Para saltar los puntos de ruptura utilizamos *Force step over*.

#### 6.1.4.2 Step into

Al ejecutar las sentencias paso a paso, también entra en la ejecución de los métodos o funciones que se encuentre. Utilizamos esta opción cuando no estamos seguros de si la ejecución del método está devolviendo un resultado correcto o no.

Para ejecutar una línea de código entrando en métodos o funciones pulsamos F7. Si hay varias invocaciones de métodos o funciones en la línea, PyCharm preguntará en qué método entrar. Esta característica se conoce como *Smart step into*.

#### 6.1.4.3 Step into my code

Cuando depuramos, podríamos querer concentrarnos en nuestro propio código y evitar que el depurador salte a clases o funciones de biblioteca. Para ello pulsamos Alt + Shift + F7.

#### 6.1.4.4 Smart step into

*Smart step into* es útil cuando hay varias invocaciones de métodos en una línea y queremos especificar sobre qué método entrar. Esta característica nos permite elegir la invocación de método en la que estamos interesados. Para ello hacemos lo siguiente:

1. Seleccionamos *Smart Step Into* desde el menú vertical o presionamos Shift + F7
2. Hacemos clic en el método. También podemos seleccionarlo con las flechas y presionar Enter.

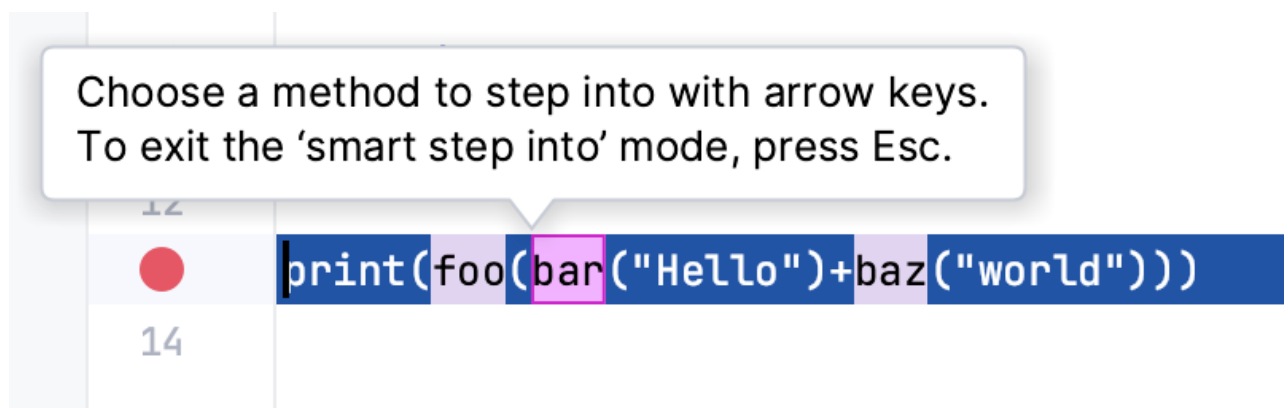


Figura 16.- Smart step into

Por defecto, *Smart step into* se emplea cada vez que hay múltiples invocaciones de métodos en una línea.

#### 6.1.4.5 Step out

Se sale de la ejecución del método actual y regresa al punto de invocación. Para ello pulsamos Shift + F8.

#### 6.1.4.6 Run to cursor

Continúa la ejecución hasta la posición del cursor. Para ello:

1. Colocar el cursor en la línea donde queremos interrumpir el programa.
2. Selección *Run to Cursor* en el menú vertical o pulsar Alt + F9.

Para saltar cualquier punto de ruptura que pudiera haber en el camino utilizamos *Force run to cursor*.

#### 6.1.4.7 Force run to cursor

Continúa la ejecución hasta la posición del cursor. Todos los puntos de ruptura se ignoran.

1. Colocar el cursor en la línea donde queremos interrumpir el programa.
2. Seleccionar *Force Run to Cursor* del menú vertical o pulsar Ctrl + Alt + F9.

#### 6.1.4.8 Force step over

Ejecuta la línea actual y va a la siguiente línea incluso si hay varias invocaciones de métodos. Si hay puntos de ruptura dentro de los métodos, se ignoran. Para ello seleccionamos *Force Step Over* del menú vertical o presionamos Alt + Shift + F8.

Cuando un punto de ruptura se alcanza o el programa se suspende, el panel *Debug* se convierte en el panel activo y nos permite obtener el control sobre la ejecución del programa. Para este propósito, podemos ejecutar los comandos de ejecución de la barra de herramientas del panel *Debug*.

#### 6.1.4.9 Jump to cursor

Para la ejecución y salta a la línea donde está situado el cursor. Todos los puntos de ruptura se ignoran.

1. Colocar el cursor en la línea donde queremos parar el programa.
2. Clic derecho y seleccionamos *Jump To Cursor* del menú contextual.

## 6.2 VS Code

Una de las características clave de VS Code es el soporte a la depuración de programas. El depurador de VS Code ayuda a acelerar el proceso de codificación y depuración.

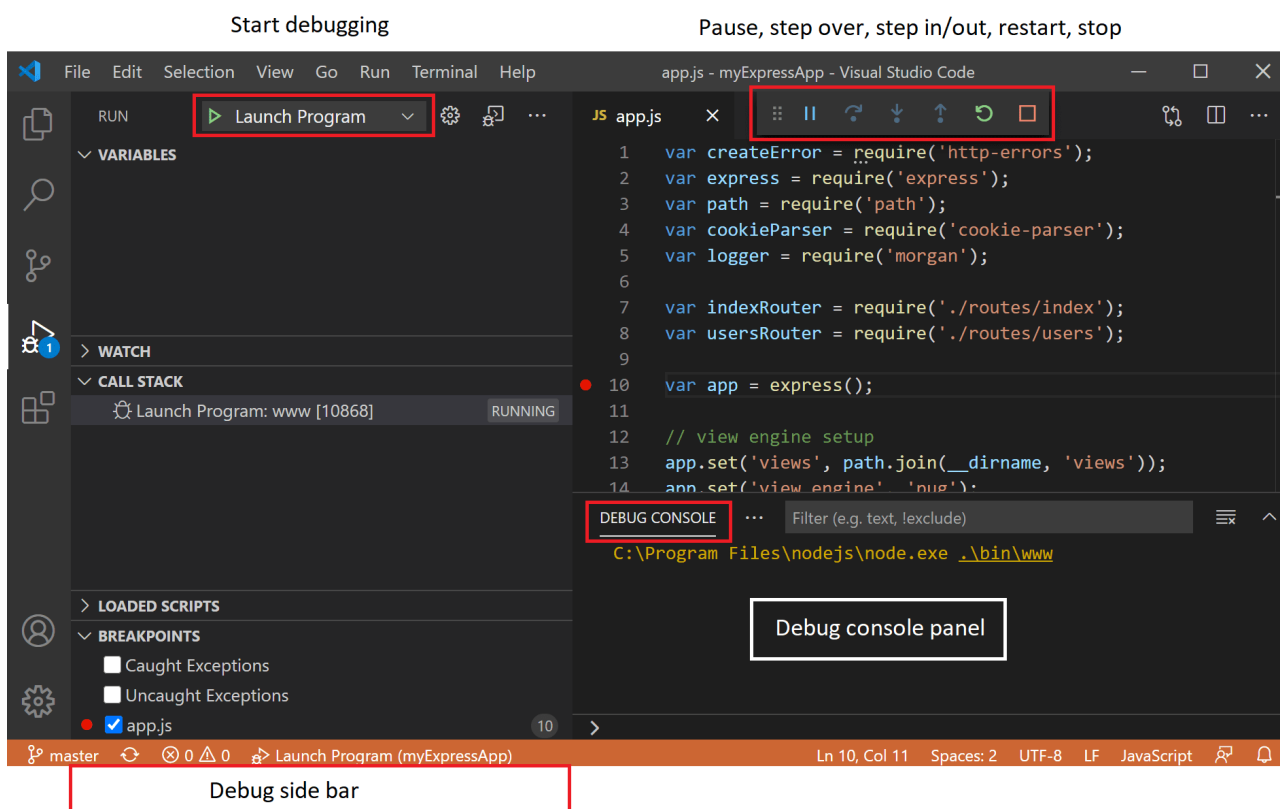


Figura 17.- Sesión de depuración con VSCode

VS Code incluye soporte por defecto para depurar aplicaciones JavaScript y TypeScript. Para otro lenguaje hay que instalar la extensión correspondiente que da soporte a la programación a ese lenguaje.

En la barra de actividad tenemos el botón *Run and debug* el cual mostrará la vista de depuración. Aquí tenemos toda la información relacionada con la ejecución y depuración del programa, y tiene en la parte superior del editor una barra de tareas con los botones para ejecutar los comandos de depuración. Además, podemos crear un archivo **launch.json** de configuración de la ejecución y depuración si fuera necesario.

Para iniciar una sesión de depuración podemos hacer una de las siguientes acciones:

- ✓ Si solo queremos depurar un script Python, la forma más simple es seleccionar la flecha abajo al lado del botón *Run* en el editor y seleccionar *Debug Python File in Terminal*.
- ✓ Hacer clic en el botón *Run and Debug* de la barra de actividad y en la vista hacer clic en el botón *Run and Debug*.

### 6.2.1 Puntos de ruptura y puntos de registro

Los puntos de ruptura se pueden establecer/borrar haciendo clic en el margen izquierdo de la línea de código o pulsando F9 en la línea actual del cursor. En la sección *BREAKPOINTS* de la vista *Run and Debug* podemos también gestionar los puntos de ruptura.

Debemos tener en cuenta lo siguiente:

- ✓ Los puntos de ruptura se muestran como círculos sólidos en rojo.
- ✓ Los puntos de ruptura deshabilitados se muestran como círculos sólidos en gris.

Los puntos de ruptura aparecerán en la sección *BREAKPOINTS* de la vista *Run and Debug*. Desde aquí podemos borrarlos haciendo clic en el botón x a la derecha de cada uno o deshabilitarlos desactivando la casilla a la izquierda del punto de ruptura.

Un punto de registro (logpoint) es una variante del punto de ruptura que no interrumpe el programa, sino que envía un mensaje log a la consola. Los puntos de registro son especialmente útiles para inyectar mensajes en programas que no pueden ser pausados o parados.

Para crear un punto de registro hacemos lo siguiente:

1. Situar el cursor en la línea del código fuente donde queremos añadir un punto de registro.
2. Seleccionar la opción de menú *Run → New breakpoint → Logpoint...*
3. Se abre un cuadro de texto para escribir un texto. Podemos incluir expresiones encerradas entre llaves.

Un punto de registro se representa por un icono en forma de rombo. Los mensajes son en texto plano pero pueden incluir expresiones para ser evaluadas entre llaves. Al igual que los puntos de ruptura, los puntos de registro se pueden habilitar o deshabilitar y pueden controlarse mediante una condición.

Una potente característica de depuración en VS Code es la posibilidad de establecer puntos de ruptura basados en condiciones. En este caso, el punto de ruptura se alcanza siempre que al evaluar la expresión el resultado sea true.

Para incluir una condición de disparo hacemos lo siguiente:

1. Seleccionar la opción de menú *Run → New breakpoint → Conditional breakpoint*.

2. Se abre un cuadro de texto para escribir la expresión lógica.

Si ya tenemos un punto de ruptura y queremos añadirle una condición para convertirlo en un punto de ruptura condicional podemos hacer una de las dos siguientes:

- ✓ Hacer clic en el botón del lápiz a la derecha del punto de ruptura de la sección BREAKPOINTS de la vista *Run and Debug*.
- ✓ Sobre el punto de ruptura en la sección BREAKPOINTS de la vista *Run and Debug* hacemos clic derecho y seleccionamos *Edit condition...*

### 6.2.2 Ejecución paso a paso

Para mostrar la vista *Run and Debug* seleccionamos el icono correspondiente en la barra de actividad. Una vez la sesión de depuración comienza, aquí veremos toda la información relacionada con la ejecución y depuración del programa. Además, veremos la barra de herramientas de depuración en la parte superior.



Figura 18.- Acciones de depuración

Desde aquí podemos utilizar las acciones de depuración:

| Acción - Teclado              | Descripción                                                                                                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pause/Continue - F5           | Continue → Reanuda la ejecución del programa hasta el siguiente punto de ruptura.<br>Pause → Inspecciona la ejecución del código en la línea actual y depura paso a paso.                     |
| Step Over – F10               | Ejecuta el siguiente método como una instrucción simple sin entrar en sus sentencias.                                                                                                         |
| Step Into – F11               | Entra en el siguiente método para seguir su ejecución paso a paso.                                                                                                                            |
| Step Out – Shift F11          | Cuando se entra en un método o subrutina, regresa al contexto de ejecución anterior completando la ejecución de las restantes sentencias del método actual como si fuera una sentencia única. |
| Restart<br>Ctrl + Shift + F11 | Termina la ejecución del programa actual y comienza de nuevo la depuración utilizando la configuración de ejecución anterior.                                                                 |
| Stop                          | Termina la ejecución del programa.                                                                                                                                                            |

### 6.2.3 Inspección de datos

Las variables pueden verse en la sección VARIABLES de la vista *Run and Debug* o situando el cursor sobre una variable en el código fuente del editor. Los valores de las variables y evaluación de las expresiones es relativa al marco de la pila seleccionado en la

sección CALL STACK.

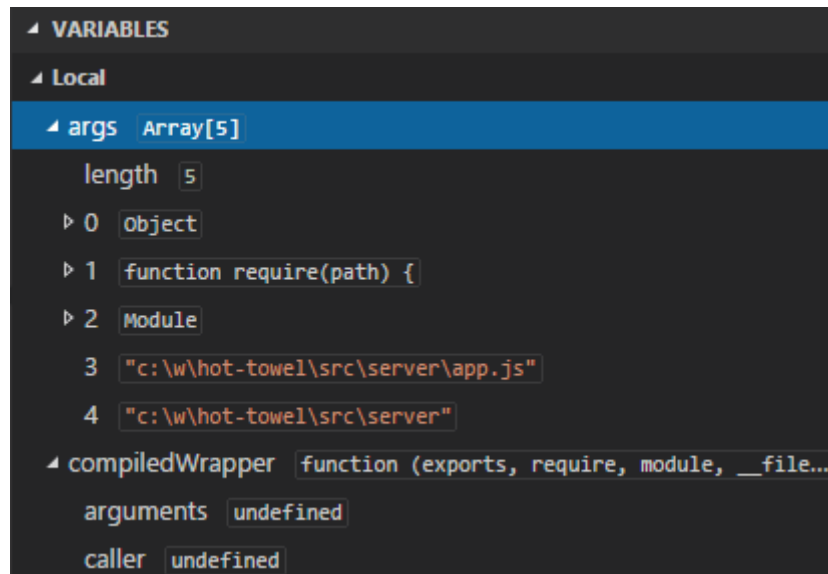


Figura 19.- Inspección de variables

Los valores de las variables pueden modificarse con la opción *Set Value* del menú contextual de las variables. Además, podemos utilizar la opción *Copy Value* para copiar el valor de la variable, o la opción *Copy as Expression* para copiar una expresión para acceder a la variable.

Las variables y las expresiones pueden también ser evaluadas y observadas en la sección WATCH de la vista *Run and Debug*.

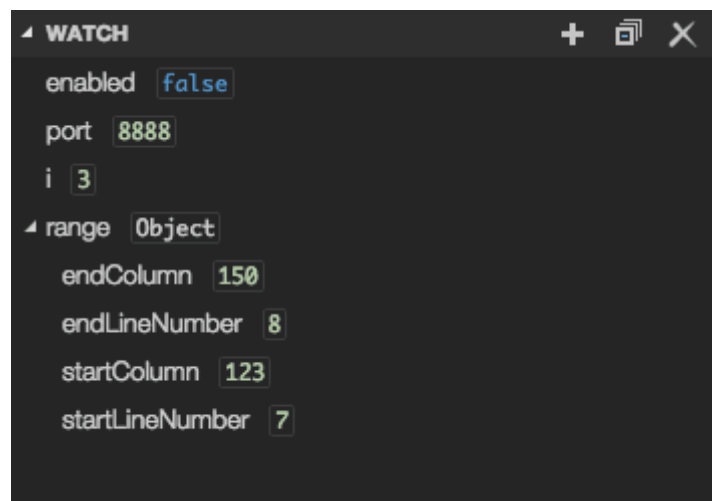


Figura 20.- Sección WATCH en Run and Debug

Para añadir una nueva observación hacemos lo siguiente:

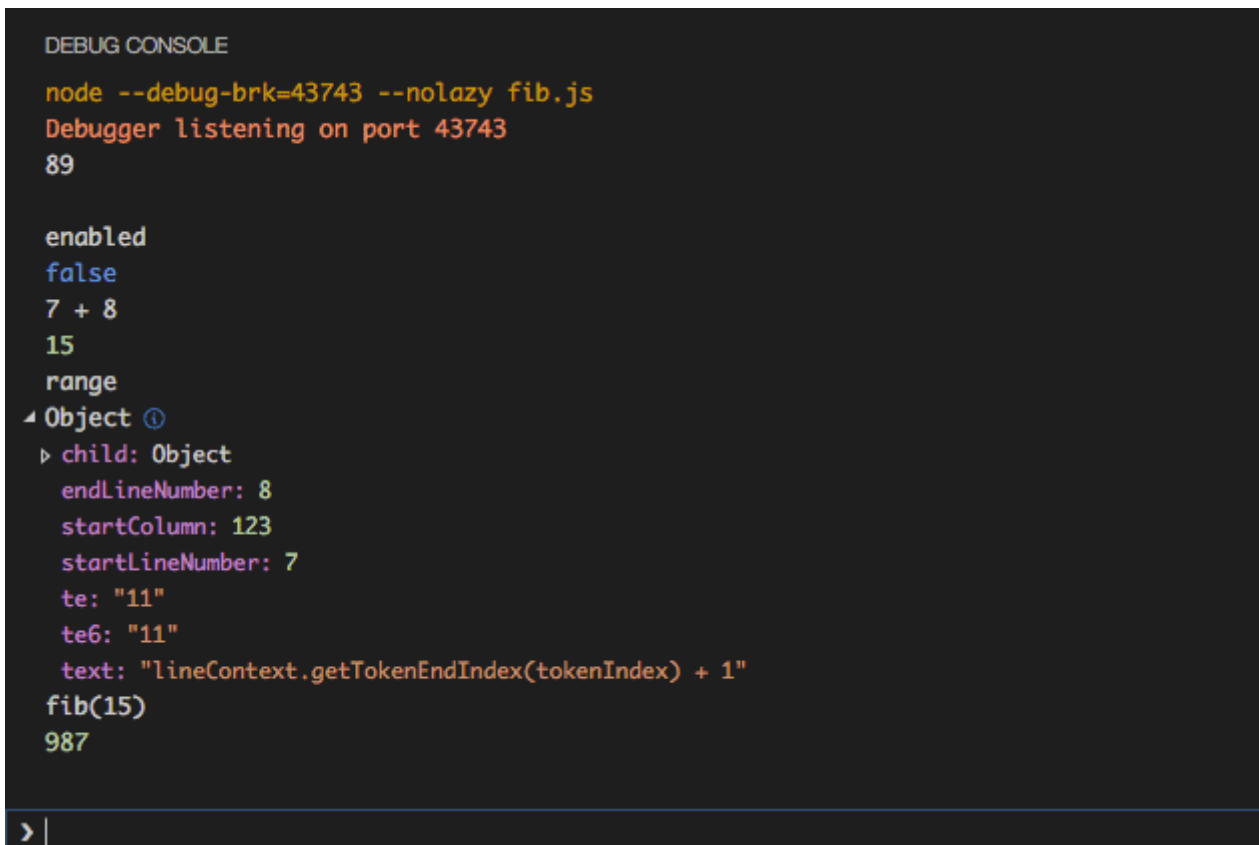
1. Hacemos clic en el botón + en la sección WATCH de la vista *Run and Debug*.
2. Escribimos la expresión a observar y pulsamos Enter.

Los nombres de variables y valores pueden filtrarse escribiéndolos cuando la sección

VARIABLES tiene el foco de teclado.

### 6.2.4 Consola depuración REPL

Las expresiones pueden evaluarse con la consola de depuración con la característica REPL (*Read-Eval-Print Loop*). Para abrir la consola de depuración utilizamos el botón *Debug Console* en la parte superior del panel *Debug*. Las expresiones se introducen en el cuadro de texto inferior y se evalúan cuando pulsamos Enter. Si necesitamos varias líneas, utilizamos Shift + Enter para crear una nueva línea y finalmente usamos Enter para que todas ellas se evalúen.



```
DEBUG CONSOLE
node --debug-brk=43743 --nolazy fib.js
Debugger listening on port 43743
89

enabled
false
7 + 8
15
range
Object
  child: Object
    endLineNumber: 8
    startColumn: 123
    startLineNumber: 7
    te: "11"
    te6: "11"
    text: "lineContext.getTokenEndIndex(tokenIndex) + 1"
fib(15)
987
> |
```

Figura 21.- Consola de depuración

La entrada de la consola de depuración utiliza el modo de editor activo, lo que significa que soporta coloración de sintaxis, sangrado, auto cierre de comillas y otras características del lenguaje.

## 7 Pruebas automatizadas con UnitTest

Las pruebas de unidad, es decir, aquellas que afectan a una clase y a sus métodos, implican crear algún objeto de la clase que se está probando y realizar invocaciones de sus métodos. Esto, puede hacerse manualmente, pero también existen herramientas que permiten realizar estas pruebas de manera automatizada.

Para las aplicaciones escritas en lenguaje Python disponemos de varias bibliotecas que

permiten realizar las pruebas de forma automática. En este capítulo veremos **unittest** como parte de la biblioteca estándar de Python.

El framework de pruebas unitarias **unittest** está inspirado inicialmente en JUnit y ofrece funcionalidad similar a los principales frameworks de pruebas unitarias que existen en otros lenguajes. Da soporte a automatización de pruebas, inicialización compartida, código de cierre de las pruebas, agregación de las pruebas en colecciones e independencia de las pruebas del framework.

Para conseguir esto, **unittest** da soporte a ciertos conceptos importantes:

- ✓ Configuración de prueba (**fixture**) → Una prueba **fixture** representa los preparativos para realizar una o más pruebas y las acciones de limpieza asociadas. Esto puede incluir, por ejemplo, la creación de bases de datos temporales, directorios o el arranque de procesos del servidor.
- ✓ Caso de prueba → Un caso de prueba es la unidad mínima de prueba. Verifica la respuesta específica a un conjunto particular de datos de entrada. **unittest** proporciona una clase base, **TestCase**, que se puede utilizar para crear nuevos casos de prueba.
- ✓ Conjunto de pruebas → Un test suite es una colección de casos de prueba, juegos de prueba o ambos. Se usa para agrupar pruebas que se han de ejecutar juntas.
- ✓ Ejecutor de pruebas → Un test runner es un componente que dirige la ejecución de las pruebas y proporciona un resultado. El ejecutor puede disponer de una interfaz gráfica, de texto o devolver un valor especial que indique el resultado de la ejecución de las pruebas.

## 7.1 Ejemplo de prueba

El módulo **unittest** proporciona un conjunto de herramientas para construir y ejecutar pruebas. Esta sección demuestra que un pequeño subconjunto de las herramientas es suficiente para satisfacer las necesidades de la mayoría.

Vamos a ver un ejemplo con el siguiente script Python que declara una clase **Numero** y varios métodos para realizar algunas operaciones con los números.

```
class Numero(object):
    def es_perfecto(self, n:int) -> bool:
        suma_divisores: int = 1
        divisor: int = int(n / 2)

        while divisor > 1:
            if n % divisor == 0:
                suma_divisores += divisor
                divisor = divisor - 1

        if n == suma_divisores:
            return True
        else:
```

```

        return False
    # Fin del método es_perfecto

    def media(self, * numeros: list[int]) -> float:
        suma: int = 0
        for n in numeros:
            suma += n
        return suma / len(numeros)
    # Fin del método media

    def mayor(self, * numeros: list[int]) -> int:
        mayor = numeros[0]
        for n in numeros:
            if n > mayor:
                mayor = n

        return mayor
    # Fin del método mayor

    def menor(self, * numeros: list[int]) -> int:
        menor = numeros[0]
        for n in numeros:
            if n < menor:
                menor = n

        return menor
    # Fin del método menor

    # Método factorial
    def factorial(self, n: int) -> int:
        if n < 0:
            raise ValueError("El número solo puede ser positivo")

        if n > 1:
            return n * self.factorial(n - 1)
        else:
            return n

    # Fin del método factorial

# Fin de la clase Numero

```

Para crear un caso de prueba se genera una subclase de **unittest.TestCase**. Cada prueba se implementa con un método cuyo nombre comienza por **test\_**. Esta convención de nombres indica al ejecutor de pruebas qué métodos representan pruebas.

En cada prueba se invoca un método **assert** para verificar el resultado de la prueba. En la clase **unittest.TestCase** disponemos de los siguientes métodos **assert**:

| Método                         | Comprueba que       |
|--------------------------------|---------------------|
| <code>assertEqual(a, b)</code> | <code>a == b</code> |

| Método                                 | Comprueba que                              |
|----------------------------------------|--------------------------------------------|
| <code>assertNotEqual(a, b)</code>      | <code>a != b</code>                        |
| <code>assertTrue(x)</code>             | <code>bool(x)</code> es <code>True</code>  |
| <code>assertFalse(x)</code>            | <code>bool(x)</code> es <code>False</code> |
| <code>assertIs(a, b)</code>            | <code>a is b</code>                        |
| <code>assertIsNot(a, b)</code>         | <code>a is not b</code>                    |
| <code>assertIsNone(x)</code>           | <code>x is None</code>                     |
| <code>assertIsNotNone(x)</code>        | <code>x is not None</code>                 |
| <code>assertIn(a, b)</code>            | <code>a in b</code>                        |
| <code>assertNotIn(a, b)</code>         | <code>a not in b</code>                    |
| <code>assertIsInstance(a, b)</code>    | <code>isinstance(a, b)</code>              |
| <code>assertNotIsInstance(a, b)</code> | <code>not isinstance(a, b)</code>          |
| <code>assertRaises(exception)</code>   | Se lanza la excepción <i>exception</i>     |

En el primero se invoca **assertEqual()** para verificar un resultado esperado; **assertTrue()** o **assertFalse()** para verificar un resultado lógico; o **assertRaises()** para asegurar que se lanza una excepción específica. Se utilizan estos métodos en lugar de la sentencia **assert** para que el ejecutor de pruebas pueda acumular todos los resultados de la prueba para realizar un informe.

A continuación se muestra un modo sencillo de ejecutar las pruebas. El método **unittest.main()** proporciona una interfaz de línea de comando para el script de prueba. Cuando se ejecuta desde la línea de comando, el script anterior produce una salida como:

```
import unittest
from Numero import Numero

class MyTestCase(unittest.TestCase):
    def setUp(self):
        self.numero = Numero()

    def test_es_perfecto(self):
        n = self.numero
        self.assertTrue(n.es_perfecto(28))
        self.assertFalse(n.es_perfecto(15))
```

```
def test_media(self):
    n = self.numero
    self.assertEqual(n.media(3, 4, 5), 4)
    self.assertEqual(n.media(32, 15, 18, 21, 35), 24.2)

    self.assertNotEqual(n.media(45, 18, 20, 31, 4), 15)

def test_mayor(self):
    n = self.numero
    self.assertEqual(n.mayor(32, 15, 18, 21, 35), 35)

def test_menor(self):
    n = self.numero
    self.assertEqual(n.menor(32, 15, 18, 21, 35), 15)

def test_excepcion(self):
    n = self.numero
    self.assertRaises(ValueError, n.factorial, -5)

def test_con_with(self):
    n = self.numero
    with self.assertRaises(ValueError):
        n.factorial(-5)

if __name__ == '__main__':
    unittest.main()
```

A continuación se muestra el resultado de ejecutar la clase **TestNumero**.

```
Ran 6 tests in 0.001s
OK
```

Repasemos las pruebas realizadas:

- ✓ El primer método de la clase de prueba se encarga de probar el método **es\_perfecto()**. Para ello utilizamos dos casos de prueba. Uno en el que se pasa como argumento un número que es perfecto y por tanto el método debe devolver **True**. En el segundo, se pasa un número que no es perfecto y debe devolver **False**. De ahí que en el primer caso utilizamos **assertTrue()** y en el segundo **assertFalse()**.
- ✓ A continuación el método **test\_media()** realiza tres pruebas para verificar si el método **media()** calcula la media aritmética de un conjunto de números. En este caso utilizamos los métodos **assertEqual()** y **assertNotEqual()**.
- ✓ Posteriormente el método **test\_mayor()** y **test\_menor()** verifican que los métodos **mayor()** y **menor()** devuelven respectivamente el mayor y el menor número de una lista. Aquí también utilizamos **assertEqual()**.
- ✓ Finalmente verificamos que un método lanza una excepción cuando se produce un error. Con **test\_exception()** se comprueba que la invocación de un método con unos argumentos concretos lanza la excepción esperada. En este caso el método

**factorial()** para un argumento negativo debe lanzar la excepción **ValueError**. Para ello usamos el método **assertRaises()**.

En este último caso también utilizamos **assertRaises()** en un gestor de contexto con el operador **with** lo que proporciona mayor claridad y legibilidad al código de prueba.

## 8 Bibliografía

---

PIÑEIRO GÓMEZ, J.M. (2022) *Entornos de Desarrollo*. 1ª Edición. Ed. Paraninfo

GSITIC (9-5-2019) *Introducción a la calidad del software*.  
<https://gsitic.wordpress.com/2019/05/09/biii17-la-calidad-del-software-y-su-medida-modelos-metricasnormas-y-estandares/>

PYTHON SOFTWARE FOUNDATION (3-12-2023) *unittest — Framework de tests unitarios*.  
<https://docs.python.org/es/3/library/unittest.html>

Microsoft Corp. (s.f.) *Visual Studio Code Docs*. <https://code.visualstudio.com/docs>

JetBrains (s.f.) *PyCharm 2023.2 Help*. <https://www.jetbrains.com/help/pycharm/getting-started.html>